ECE462L Fall Report

Tanya Nikam	Tyler Stephens	Alex Jang
TN22697	TMS4594	ACJ2662

October 16, 2025

Contents

1 Introduction to Lab 1

For the first lab in Power Electronics, the key objectives were to get familiar with the C2000. This microcontroller will control the switching of the DC-DC boost converter, as well as process current sensor data from the converter.

In Lab 1 (Week 1), the goals were to import a new C2000 demo project, generate pulse width modulation (PWM) waveforms on the C2000 and capture them on an oscilloscope, and implement dead-time between complementary PWM signals to prevent shoot-through. We learned about PWM waves and duty cycles through the lens of the different registers in the C2000 software.revent shoot-through.

In Lab 1 (Week 2), the project continued by calculating an accurate voltage estimate based on the ADC register values and converting analog to digital values using the ADC. We gained a better understanding of how the microcontroller processes input, and how to implement ADC capability in the C2000 software.

2 Pulse-Width Modulation

2.1 Table of register values

Table 1: Register Values for PWM

Register	Value
EPwm1Regs.ETSEL.bit.SOCAEN	0
EPwm1Regs.ETSEL.bit.SOCASEL	4
EPwm1Regs.ETPS.bit.SOCAPRD	1
EPwm1Regs.TBPRD	375
EPwm1Regs.CMPA.bit.CMPA	188
EPwm1Regs.TBCTL.bit.CLKDIV	0
EPwm1Regs.TBCTL.bit.HSPCLKDIV	0
EPwm1Regs.TBCTL.bit.CTRMODE	2
EPwm1Regs.AQCTLA.bit.CAD	2
EPwm1Regs.AQCTLA.bit.CAU	1
EPwm1Regs.AQCTLA.bit.ZRO	0
EPwm1Regs.AQCTLA.bit.PRD	0
EPwm1Regs.AQCTLB.bit.CAD	1
EPwm1Regs.AQCTLB.bit.CAU	2
EPwm1Regs.AQCTLB.bit.ZRO	0
EPwm1Regs.AQCTLB.bit.PRD	0
GpioCtrlRegs.GPAGMUX1.bit.GPIO0	0
GpioCtrlRegs.GPAMUX1.bit.GPIO0	1
GpioCtrlRegs.GPAGMUX1.bit.GPIO1	0
GpioCtrlRegs.GPAMUX1.bit.GPIO1	1
EPwm1Regs.DBCTL.bit.IN_MODE	0
EPwm1Regs.DBCTL.bit.POLSEL	2
EPwm1Regs.DBCTL.bit.OUT_MODE	3
EPwm1Regs.DBFED.bit.DBFED	15
EPwm1Regs.DBRED.bit.DBRED	15

2.2 Screenshots of PWMs

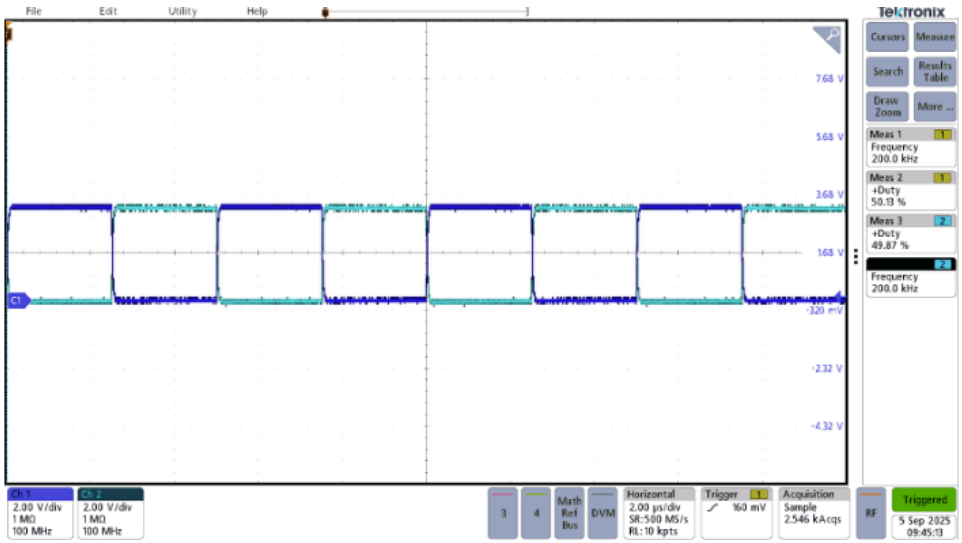

Figure 1: PWM Waveform 1: 200 KHz, 50% Duty cycle

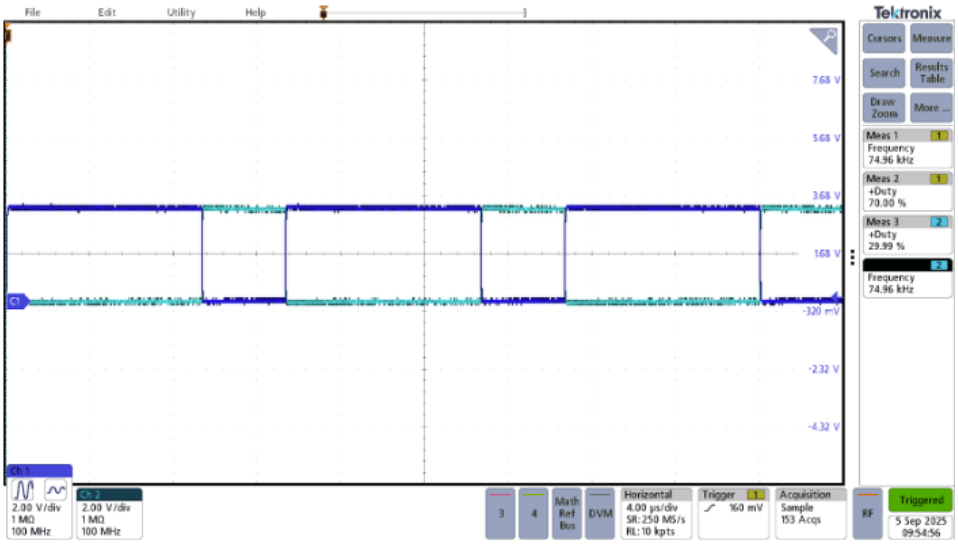

Figure 2: PWM Waveform 2: 200 KHz, 70% Duty cycle

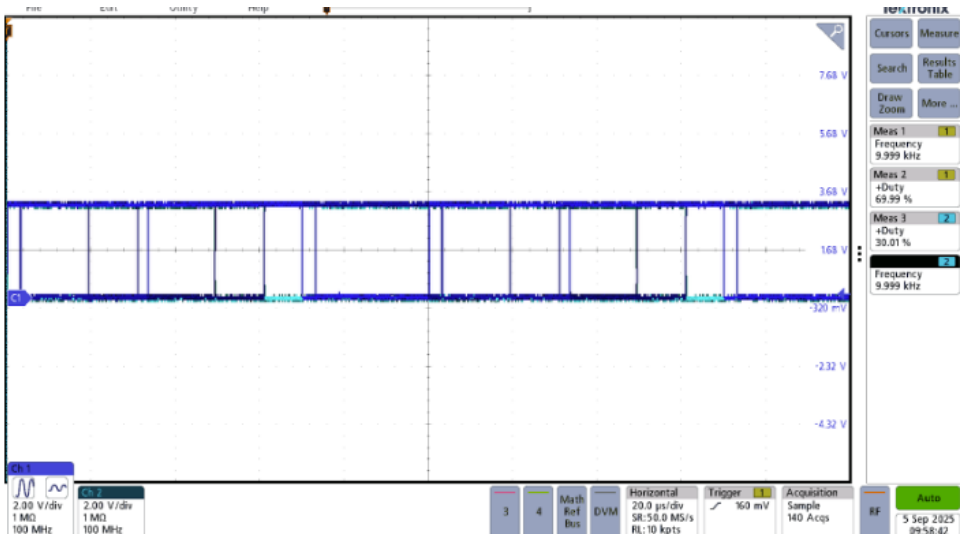

Figure 3: PWM Waveform 3: 75 KHz, 70% Duty cycle

The waveforms in Figures 1, 2, and 3 are up-down, up, and down counters respectively. It is observed that, depending on the type of counter, as Nr goes up, then there are more possible duty cycles. Duty cycles and frequency are shown in the PWM figures and show this relationship. The registers highlighted in yellow in Table 1 are set to generate these 3 different PWMs.

2.3 Screenshots of dead-time

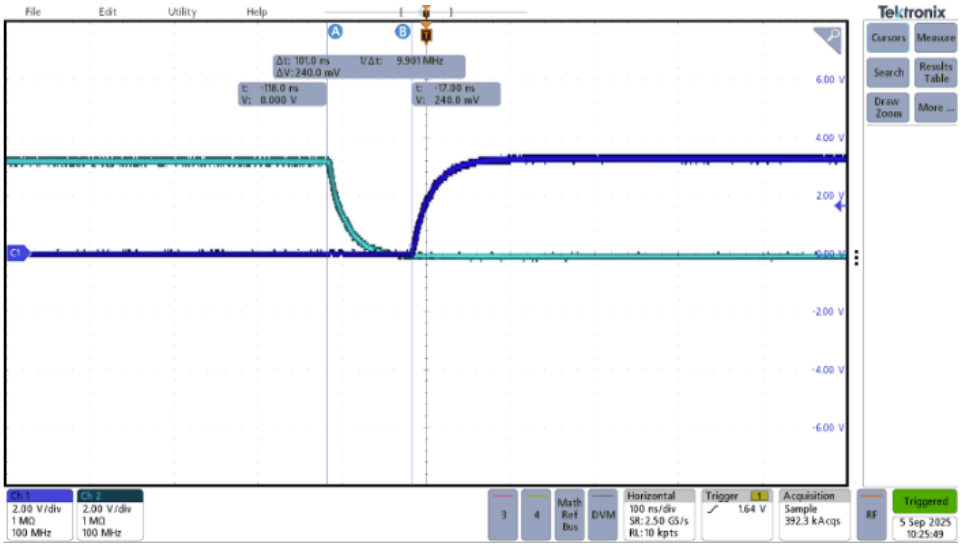

Figure 4: Rising edge

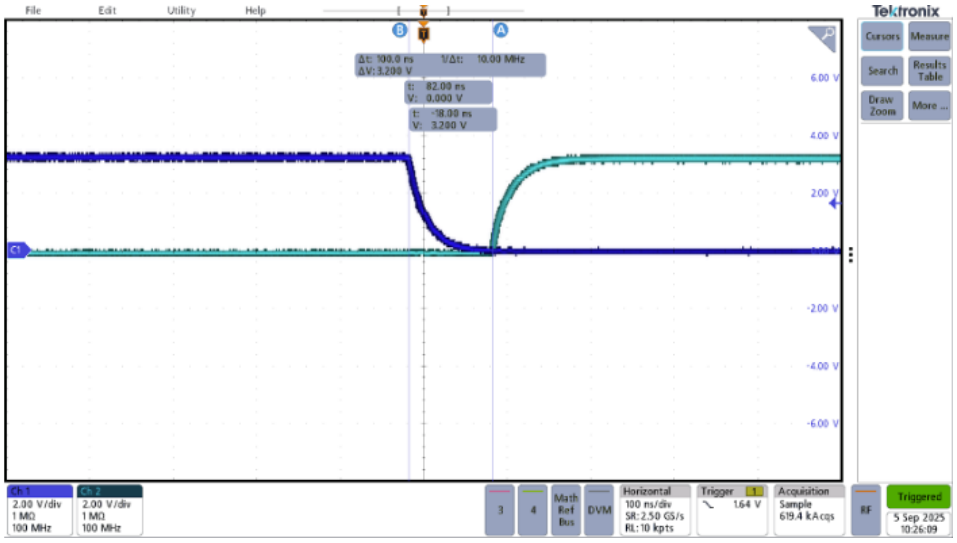

Figure 5: Falling edge

The rising and falling edge of the dead-time are observed in Figures 4 and 5, and show what happens as the pulse-width modulation changes. The Registers highlighted in green in Table 1 are configured to allow a dead-time of 100ns.

2.4 Lab 1 Week 1 Takeaways

We learned all about the different counter methods in relation to creating PWM signals and duty cycle. Now we understand the difference between the three different count methods:symmetric triangular carrier, leading edge sawtooth,and trailing edge sawtooth. These are notated as up-down count, up-count, and down-count respectively. We also have gained a greater intuitive understanding of how the step counter influences the duty cycles depending on what count is utilized. Having a higher counter value allows for increased resolution, leading to more possible different duty cycles. We also better understand how this connects back to our boost converter. Duty cycle determines the behavior of a converter like we saw starting in homework two, and having a way to control it is very important. It is also important to leave dead-time to prevent short circuiting if both switches are on at the same time. This is due to our boost converter being a synchronous converter that uses two MOSFETs, rather than an asynchronous converter with a MOSFET and a diode.

The main struggle has been understanding the bigger picture. Syntax errors and keeping track of registers can make it hard to connect it all back to our boost converter. We found success when we looked at the concepts from a higher level perspective, gaining the intuitive understanding behind counters first before getting lost in the code. This helped us find the meaning when interpreting the data.

3 Analog-to-Digital Conversion

3.1 Register Values

Table 2: Register Values for Analog-to-Digital Conversion

Register	Value
PieCtrlRegs.PIEIER1.bit.INTx1	1
CpuSysRegs.PCLKCR0.bit.TBCLKSYNC	1
AdcaRegs.ADCCTL2.bit.PRESCALE	0
AdcaRegs.ADCCTL1.bit.INTPULSEPOS	1
AdcaRegs.ADCCTL1.bit.ADCPWDNZ	1
EPwm1Regs.ETSEL.bit.SOCAEN	1
EPwm1Regs.ETSEL.bit.SOCASEL	3
EPwm1Regs.ETPS.bit.SOCAPRD	1
EPwm1Regs.ETPS.bit.INTPRD	1
EPwm1Regs.ETSOCPS.bit.SOCAPRD2	10
EPwm1Regs.ETPS.bit.SOCPSSEL	1
EPwm1Regs.TBPRD	375
EPwm1Regs.CMPA.bit.CMPA	188
EPwm1Regs.TBCTL.bit.CLKDIV	0
EPwm1Regs.TBCTL.bit.HSPCLKDIV	0
EPwm1Regs.AQCTLA.bit.CAD	2
EPwm1Regs.AQCTLA.bit.CAU	1
EPwm1Regs.AQCTLA.bit.ZRO	0
EPwm1Regs.AQCTLA.bit.PRD	0
EPwm1Regs.AQCTLB.bit.CAD	1
EPwm1Regs.AQCTLB.bit.CAU	2
EPwm1Regs.AQCTLB.bit.ZRO	0
EPwm1Regs.AQCTLB.bit.PRD	0
GpioCtrlRegs.GPAGMUX1.bit.GPIO0	0
GpioCtrlRegs.GPAMUX1.bit.GPIO0	1

Table 3: Register Values for Analog-to-Digital Conversion Continued

Register	**Value**
GpioCtrlRegs.GPAGMUX1.bit.GPIO1	0
GpioCtrlRegs.GPAMUX1.bit.GPIO1	1
EPwm1Regs.DBCTL.bit.IN$_M ODE$	0
EPwm1Regs.DBCTL.bit.POLSEL	2
EPwm1Regs.DBCTL.bit.OUT$_M ODE$	3
EPwm1Regs.DBFED.bit.DBFED	15
EPwm1Regs.DBRED.bit.DBRED	15
AdcaRegs.ADCSOC0CTL.bit.CHSEL	1
AdcaRegs.ADCSOC0CTL.bit.ACQPS	14
AdcaRegs.ADCSOC0CTL.bit.TRIGSEL	5
AdcaRegs.ADCINTSEL1N2.bit.INT1SEL	0
AdcaRegs.ADCINTSEL1N2.bit.INT1E	1
AdcaRegs.ADCINTFLGCLR.bit.ADCINT1	1
AdcaRegs.ADCINTFLGCLR.bit.ADCINT1	1
AdcaRegs.ADCINTOVFCLR.bit.ADCINT1	1
AdcaRegs.ADCINTFLGCLR.bit.ADCINT1	1

3.2 Table and Watch Window for Sampling Once

Table 4: Sampling Once

Voltage	Voltage Divider Voltage	Calculated Capacitor Voltage	Measured Capacitor Voltage	Register Values	Expected
5V	3.0V	2.8V	2.827V	3553	3722
4V	2.4V	2.415V	2.26V	2997	2978
3V	1.8V	1.709V	1.69V	2122	2233
2V	1.2V	1.177V	1.13V	1461	1489
1V	0.6V	0.5921V	0.57V	735	744
0V	0.0V	0.0313V	0.013V	38	0

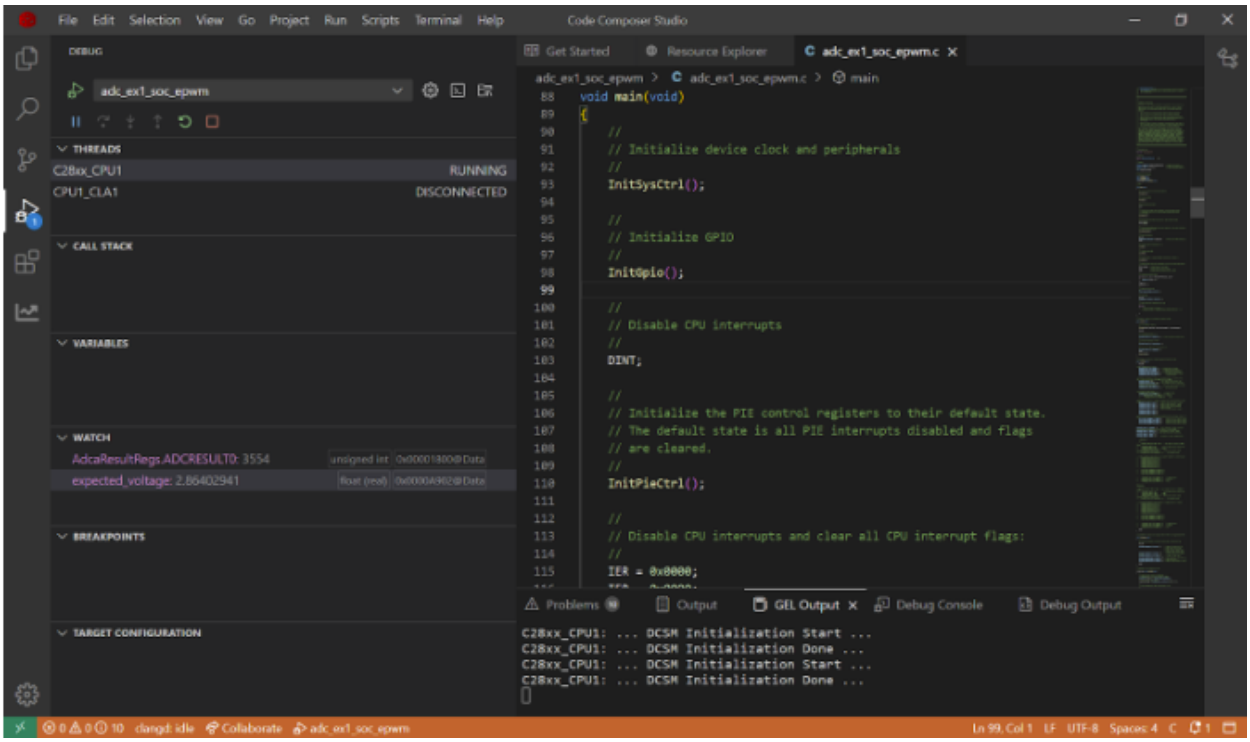

Figure 6: Screenshot for Sampling Once at 5V

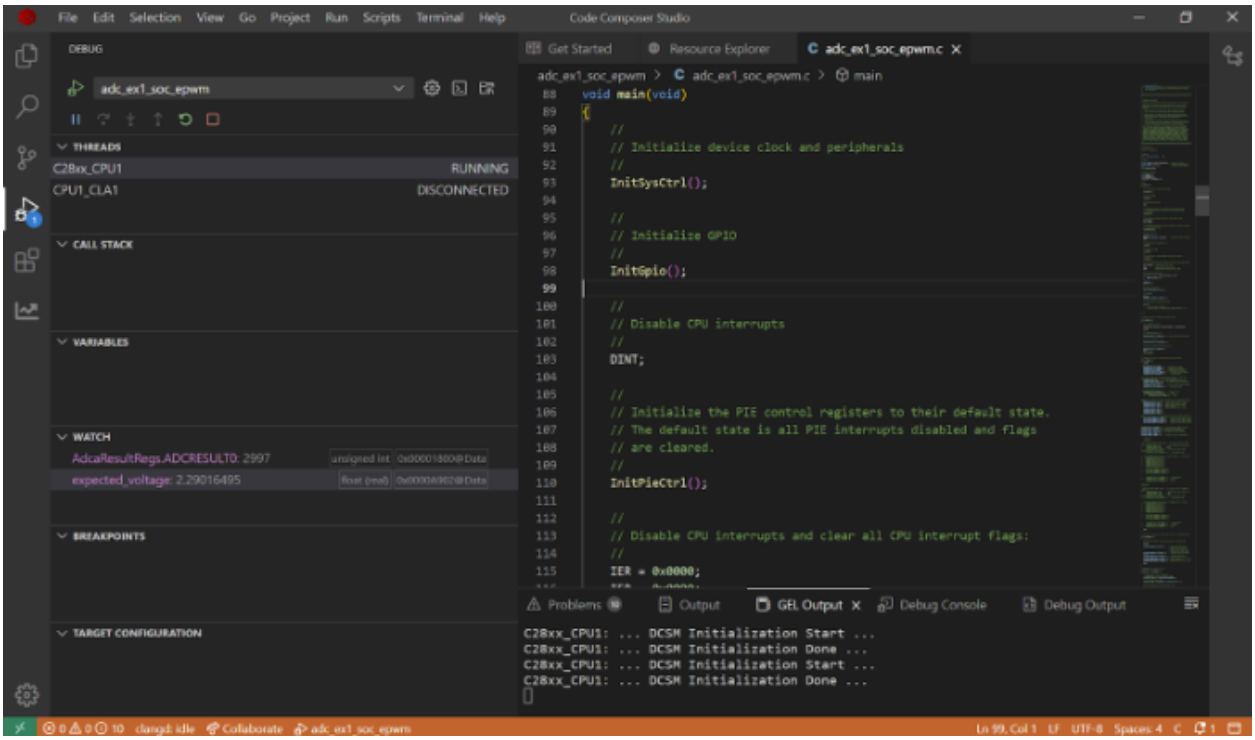

Figure 7: Screenshot for Sampling Once at 4V

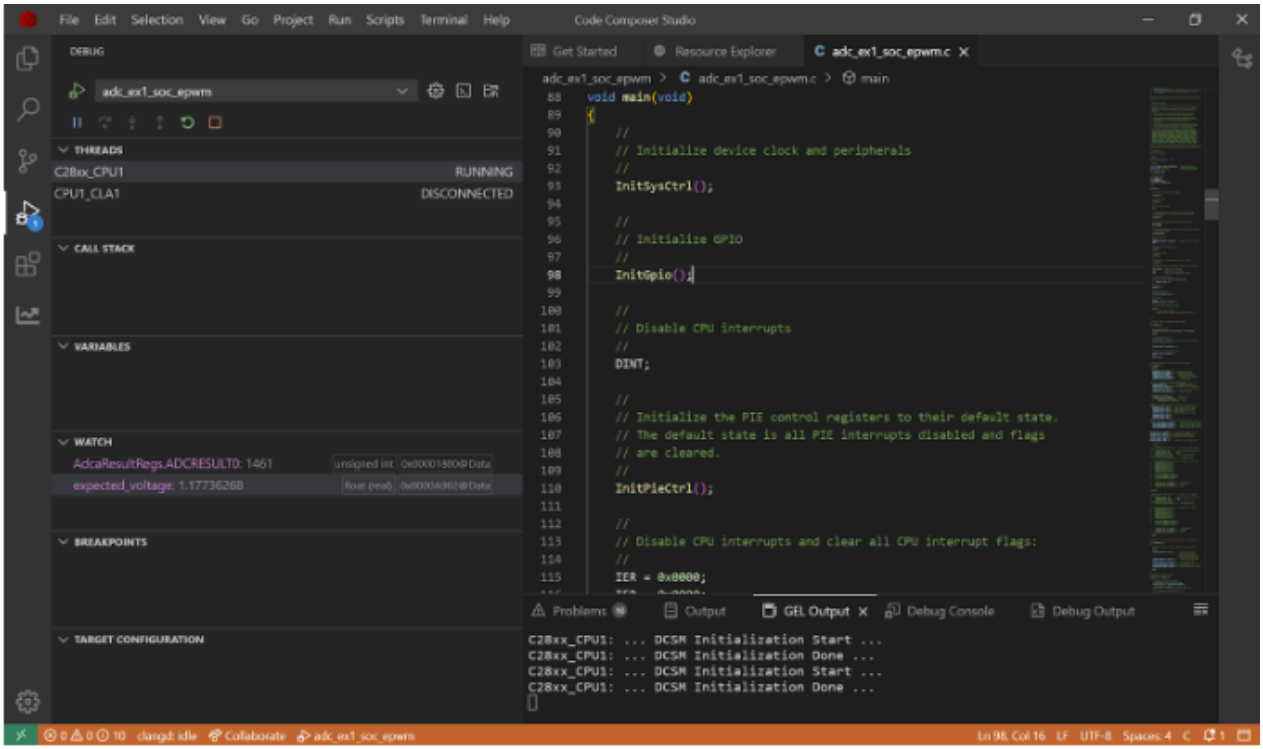

Figure 8: Screenshot for Sampling Once at 3V

3.3 Table and Watch Window for Sampling Twice

Table 5: Sampling Twice

Voltage	Voltage Divider Voltage	Calculated Capacitor Voltage	Measured Capacitor Voltage	Register Values	Expected
5V	3.0V	2.8V	2.82V	3554	3722
4V	2.4V	2.415V	2.26V	2997	2978
3V	1.8V	1.709V	1.693V	2121	2233
2V	1.2V	1.777V	1.126V	1461	1489
1V	0.6V	0.5921V	0.567V	730	744
0V	0V	0.0313V	0.013V	41	0

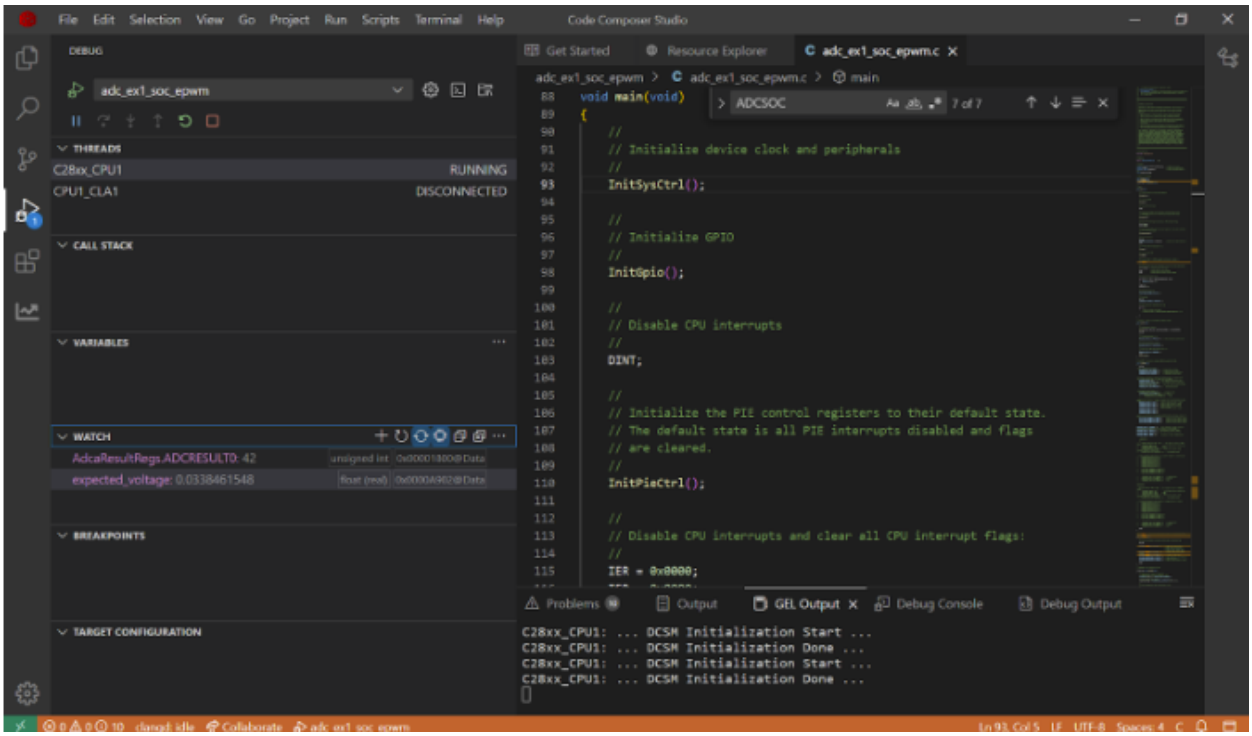

Figure 9: Screenshot for Sampling Twice at 0V

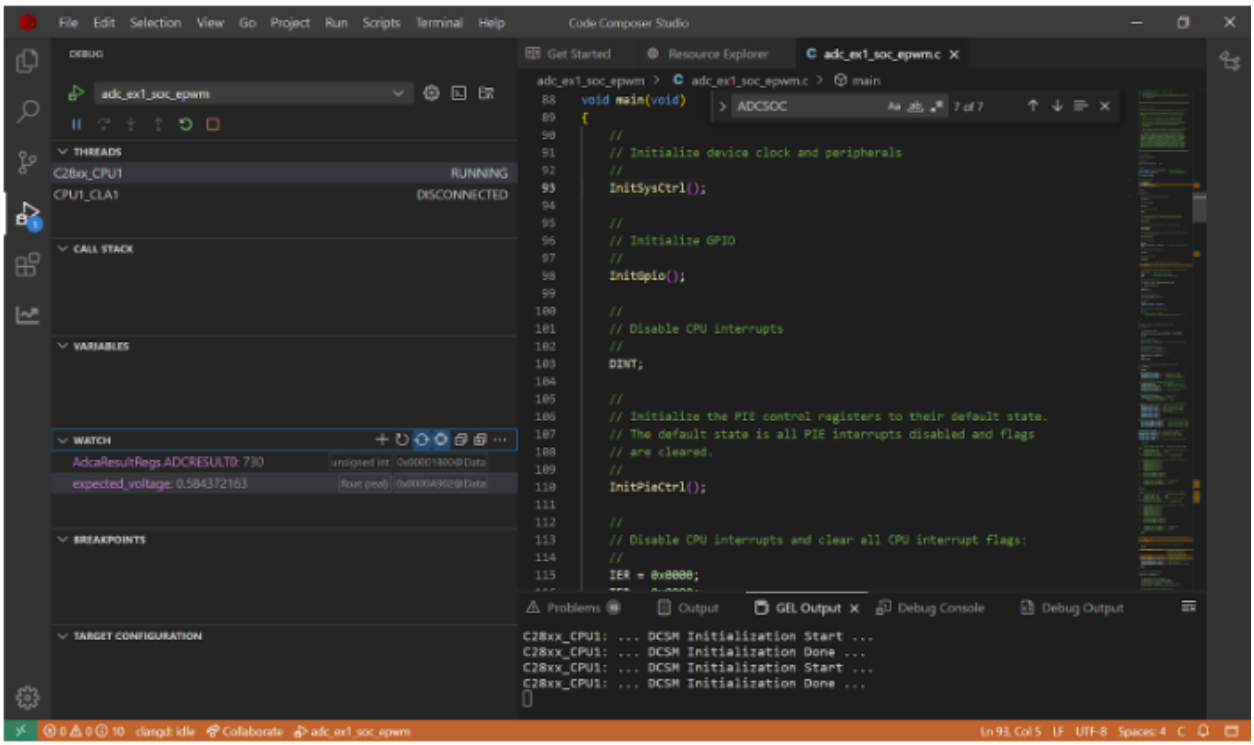

Figure 10: Screenshot for Sampling Twice at 1V

3.4 Table and Watch Window for Sampling Five Times

Table 6: Sampling Five Times

Voltage	Voltage Divider Voltage	Calculated Capacitor Voltage	Measured Capacitor Voltage	Register Values	Expected
5V	3.0V	3.00V	2.98V	3729	3722
4V	2.4V	2.41V	2.38V	2999	2978
3V	1.8V	1.8V	1.79V	2241	2233
2V	1.2V	1.21V	1.2V	1503	1489
1V	0.6V	0.61V	0.6V	763	744
0V	0V	0.02V	0.01V	22	0

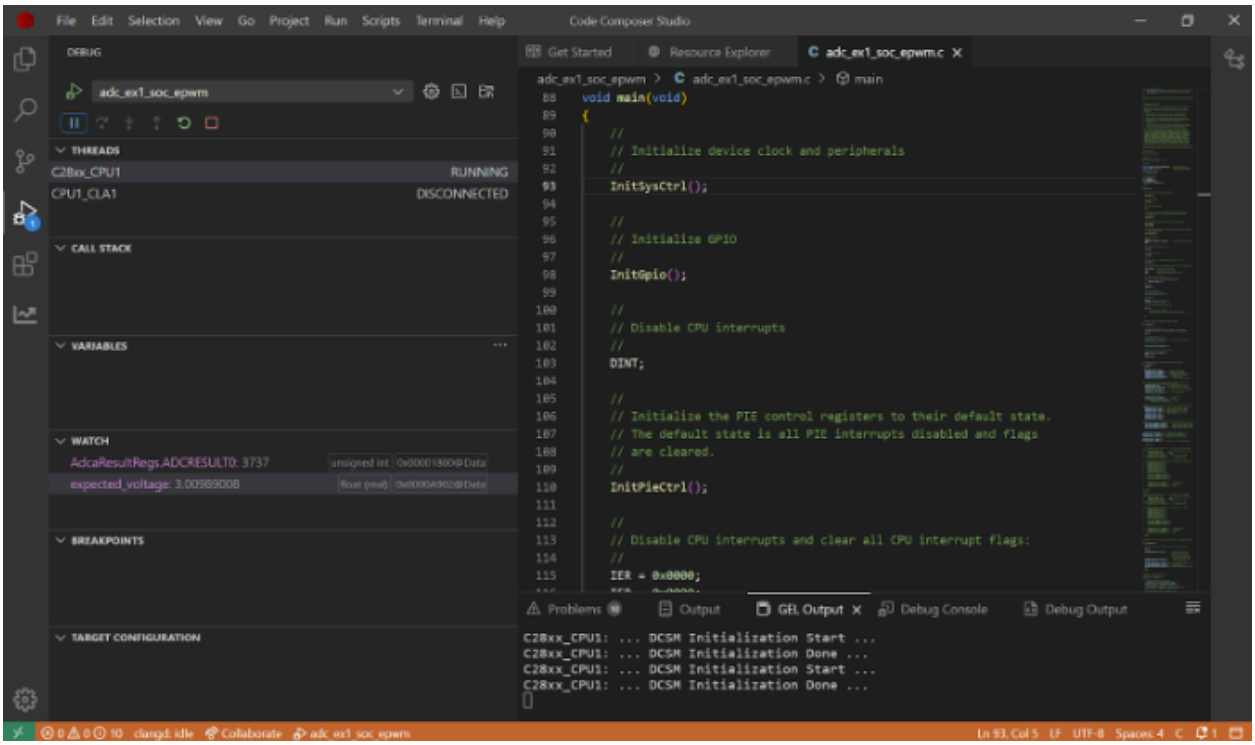

Figure 11: Screenshot for Sampling Five Times at 5V

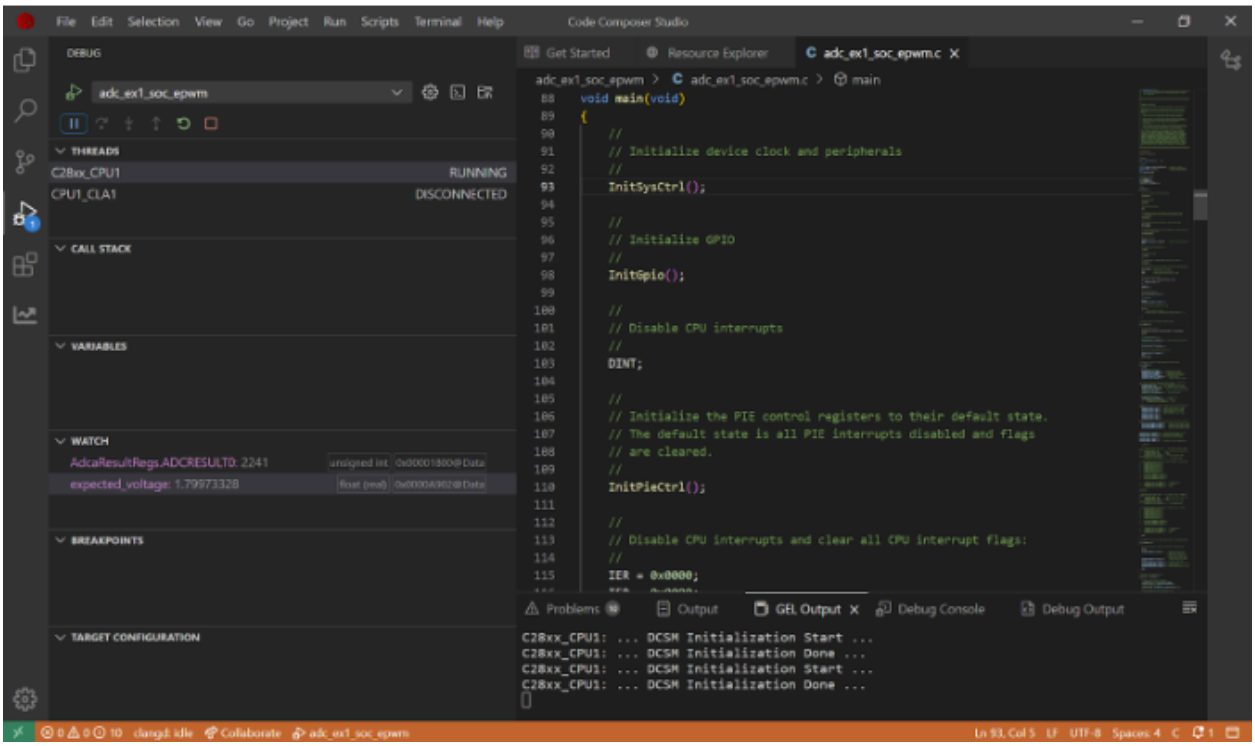

Figure 12: Screenshot for Sampling Five Times at 3V

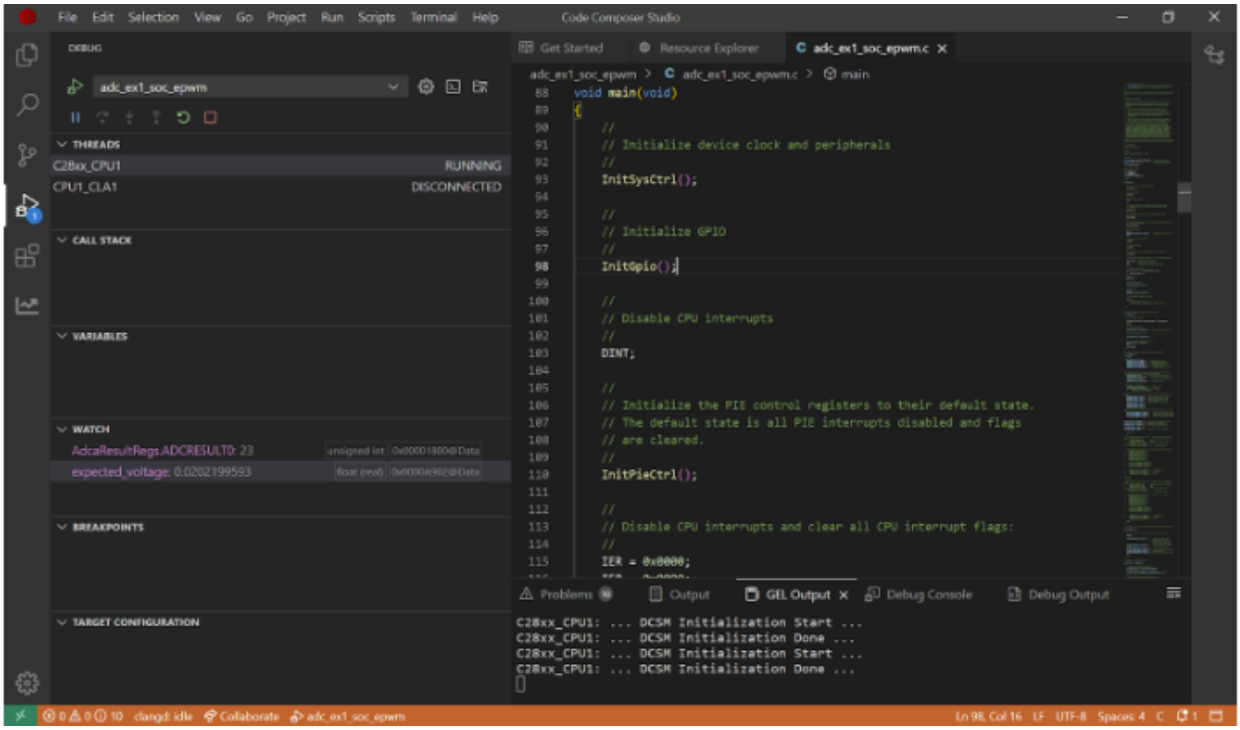

Figure 13: Screenshot for Sampling Five Times at 0V

3.5 Code recalculating voltage from ADC register

Listing 1: Code recalculating voltage from ADC register

```
1 // back calculate the voltage
2 expected_voltage = (AdcaResultRegs.ADCRESULT0 / 4095.0) * 3.3;
```

3.6 Lab 1 Week 2 Interpretations and Takeaways

As shown in Listing 1, the code calculates the expected voltage by multiplying 3.3 by the ratio of the raw amount of register values the ADC is outputing divided by 4095. It is obversed with the Tables 4,5,6 and the Figures 6, 7, 8, 9, 20, 11, 12, and 13 that the higher the sampling rate, the closer measured voltages are compared to calculated voltage. The registers highlighted blue in Tables 2 and 3 were utilized to configure the ADCs, SOCs, and change the sampling frequencies.

What you learned/struggled with/succeeded at: What was learned in this particular lab is that sampling rate can allow us to measure more accurate voltages. What the group struggled with was conceptualizing the equations needed to find the value of the EPwm1Regs.ETSOCPS.bit.SOCAPRD2 register as well as getting our ADC register to display a value. What was successful was recording quality voltages at the end and relating it to the calculated voltages.

4 Introduction to Lab 2

The objective of lab 2 to was to work on the schematic and PCB design of the 24V to 48V Boost Converter. We presented our initial design to the class, then implemented feedback for an even more optimized layout, which we submitted for production.

5 Printed Circuit Board Layout

5.1 Schematic

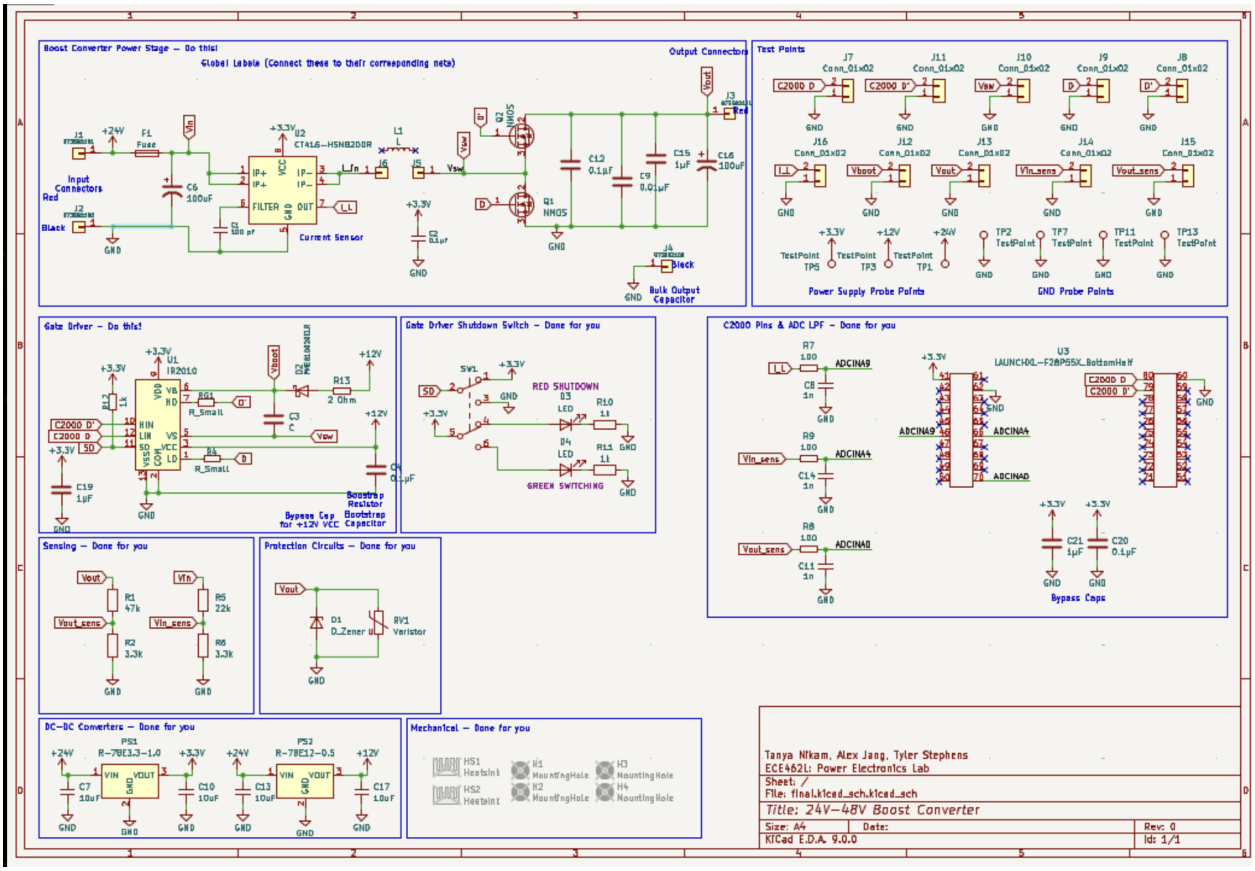

Figure 14: Overall Schematic

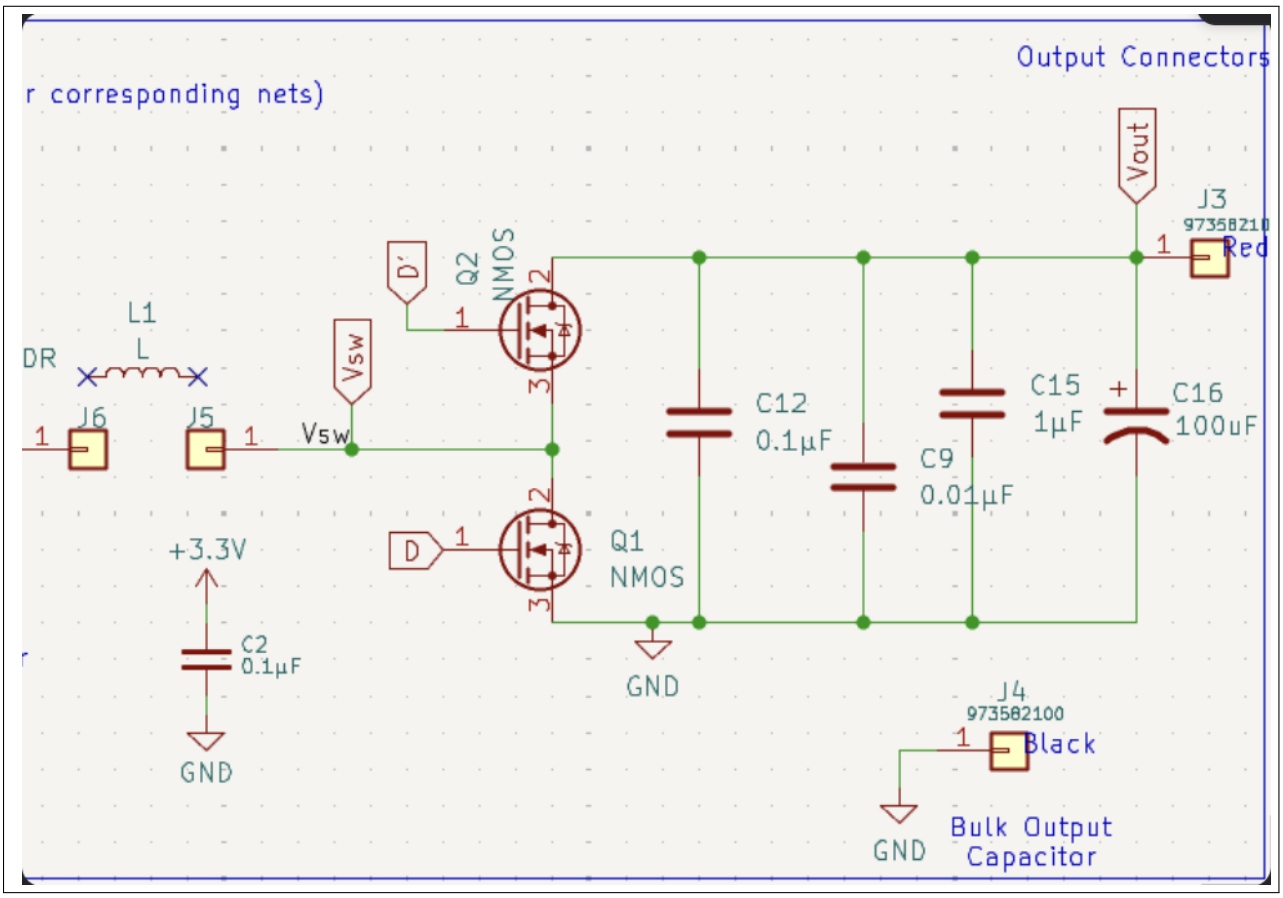

Figure 15: Schematic of the Critical Loop

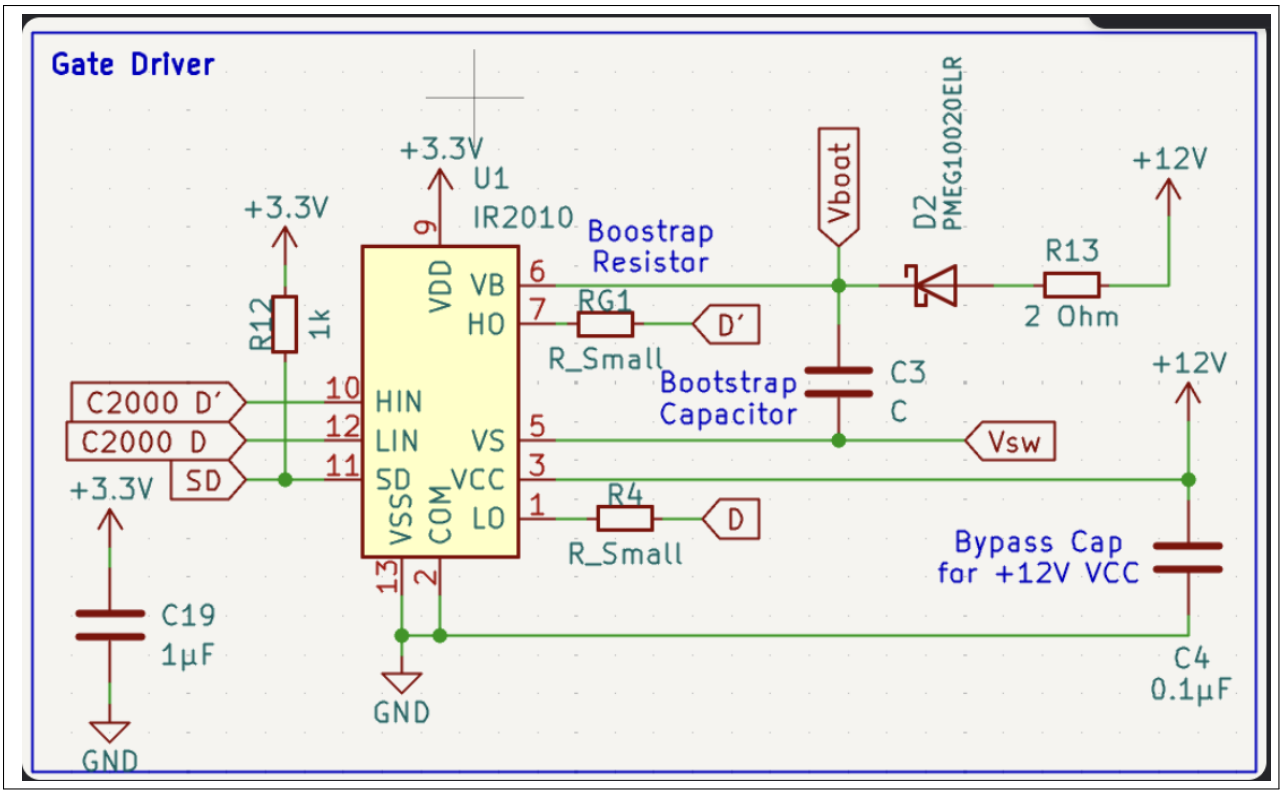

Figure 16: Schematic of the Gate Driver

5.2 Screenshots of Each Layer

At a high level, our converter contains various layers, pours, and important loops. The left half of our board has the C2000 microcontroller on top and the inductor bobbin right underneath that. Working down the right side of our board, we have a couple of testing points, our switch, and then our H-Bridge circuit that contains the critical/gating loops right next to the MOSFETS, heatsinks, and power supplies. The right edge has all our jumpers for input and output, and the bottom right corner has more testing points. The layout of the PCB is based on our schematic of the circuit, Figure 14, our critical loop, Figure 15, and our Gate Driver, Figure 16.

We utilized all four layers of our PCB as shown in Figures 18, 19, 20, and 21 below to spread out our pours, separate different signals, and minimize the size of our loops for an organized and compact layout. The front layer contains our Vout and 48V pours as well as most of the ADC signals. Multiple components, including the heat sink, power supplies, fuse, microcontroller, inductor, switch, and multiple SMD components will be soldered on the top layer. Our second layer is the ground plane, which we designed to be continuous to ensure low impedance return paths for our signals. The third layer contains a 3.3V pour that connects to the microcontroller, current sensor, switch, and other signals. Finally, the back layer contains a 12V pour, a Vin Pour, a couple of sensing signals, and certain components from the critical and gating loops. The physcial componets of everything mentioned above, as well as the microcontroller are labeled and layed out accordingly as shown in Figure 17.

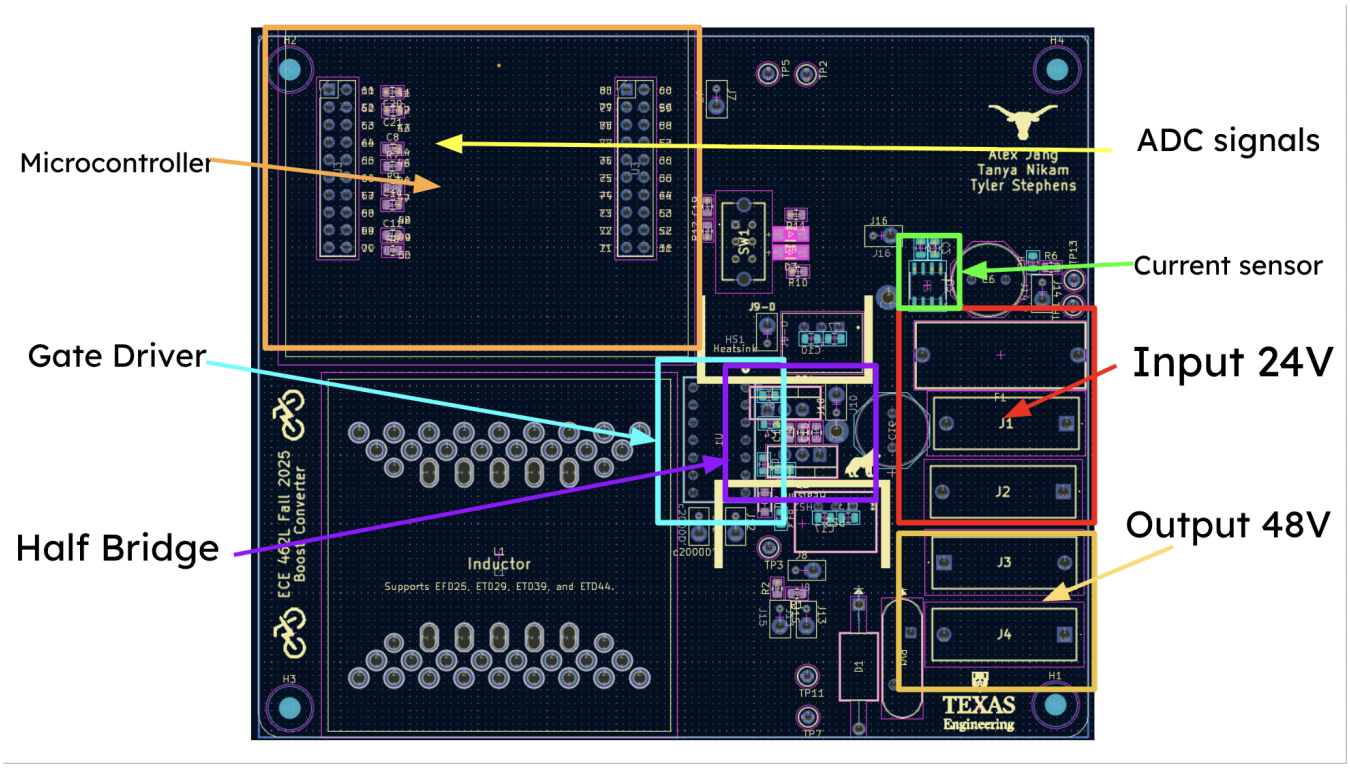

Figure 17: Labeled Layout of the PCB

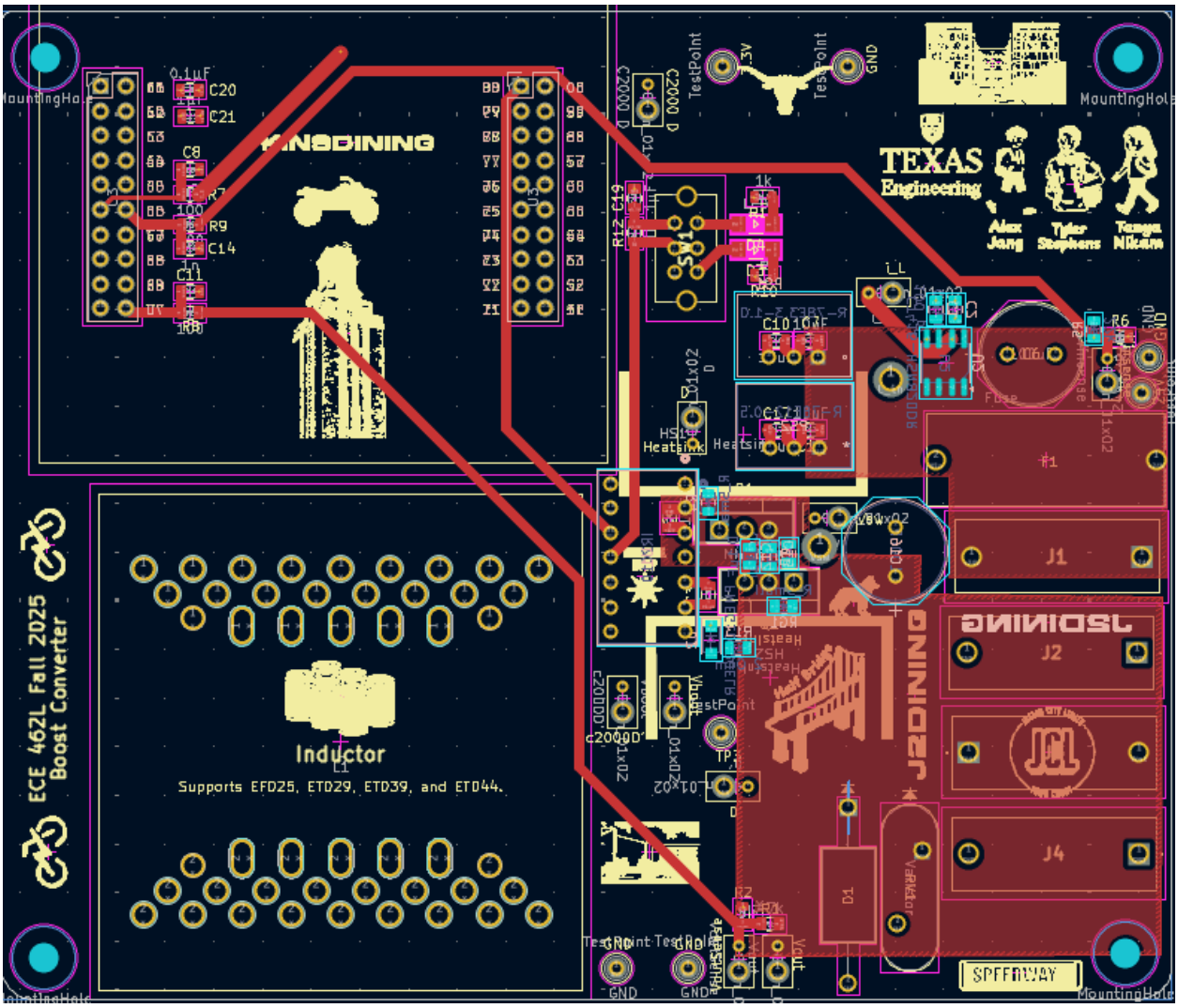

Figure 18: Top Layer (F.Cu)

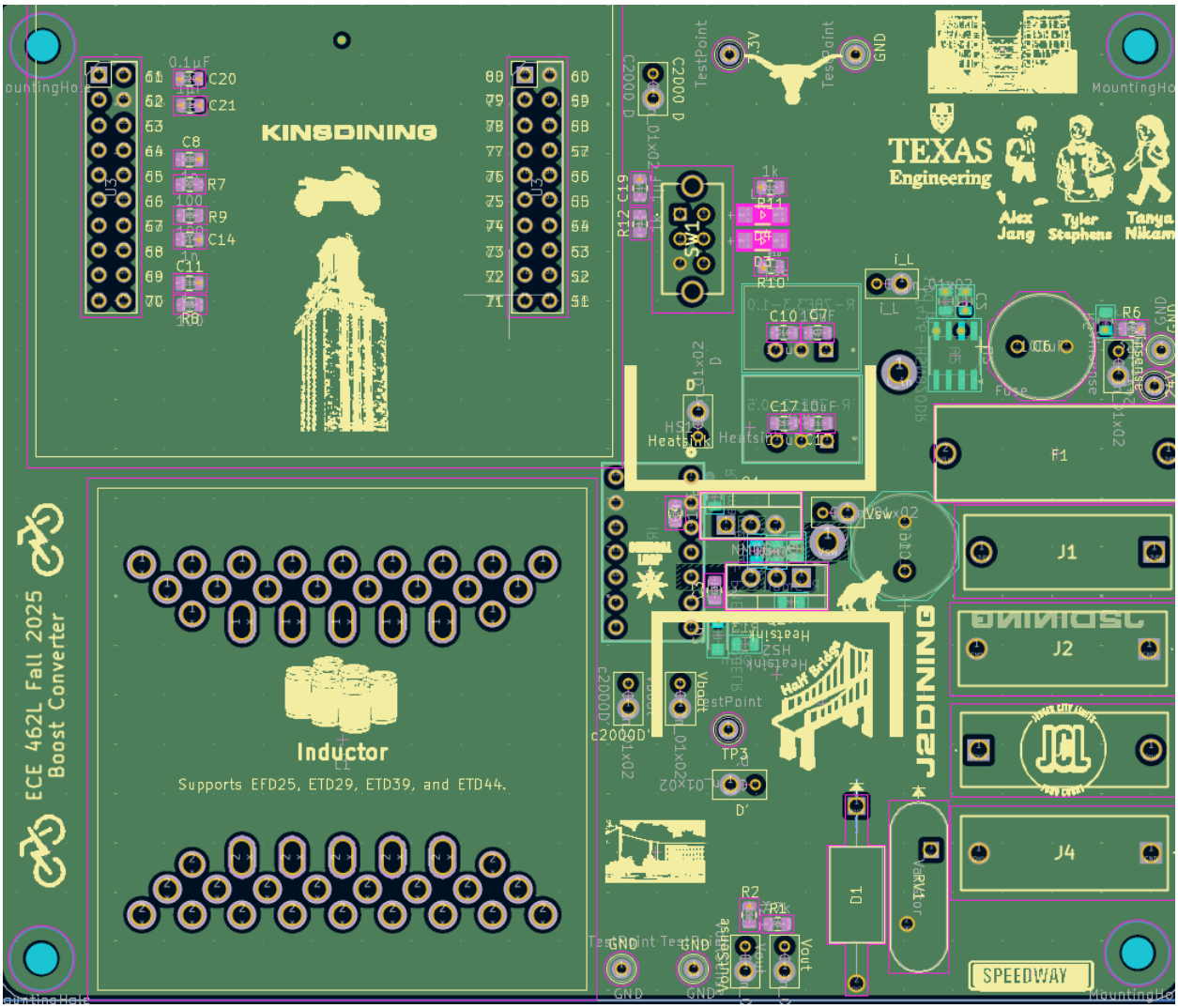

Figure 19: Inner Layer 1 (In1.Cu)

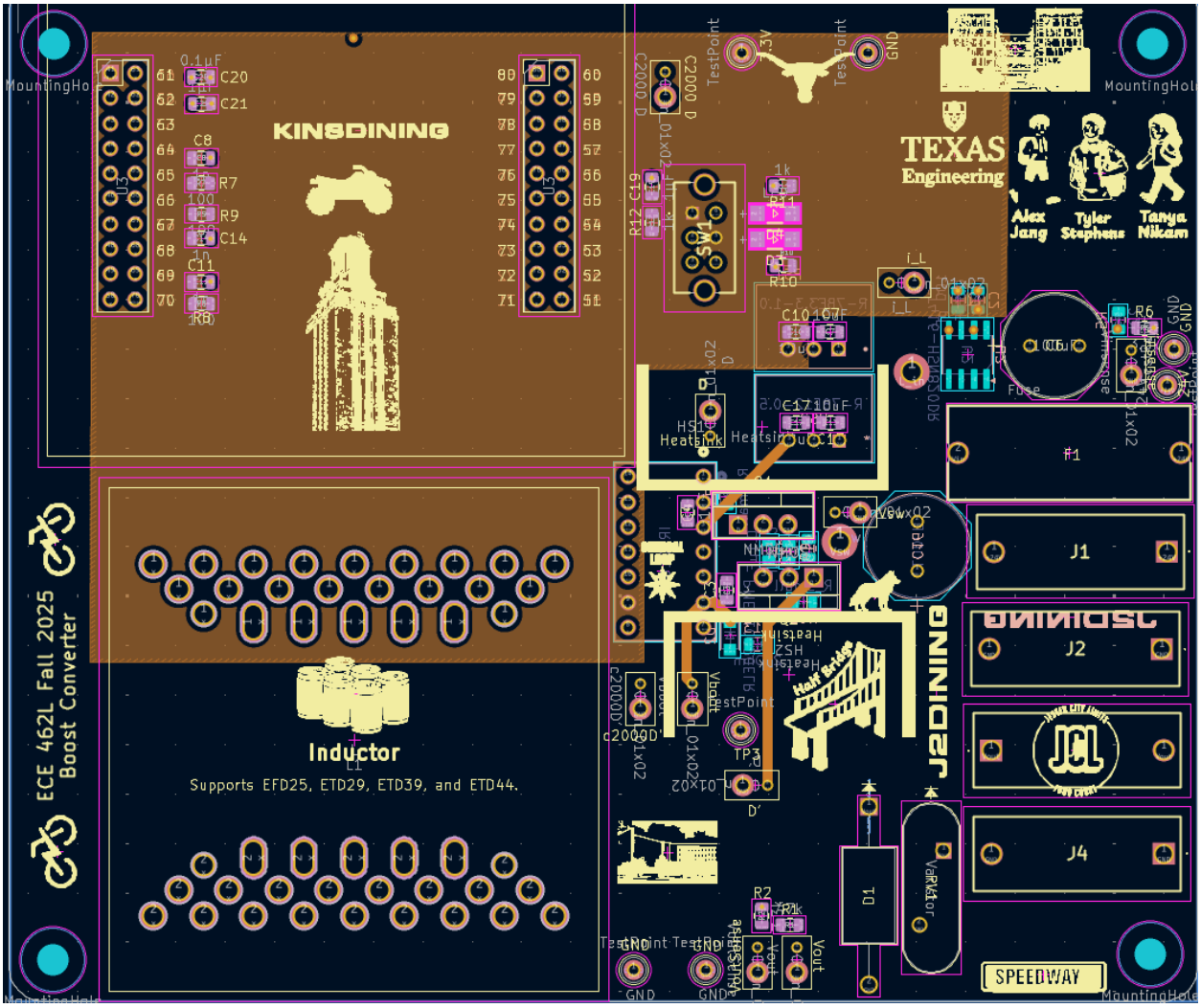

Figure 20: Inner Layer 2 (In2.Cu)

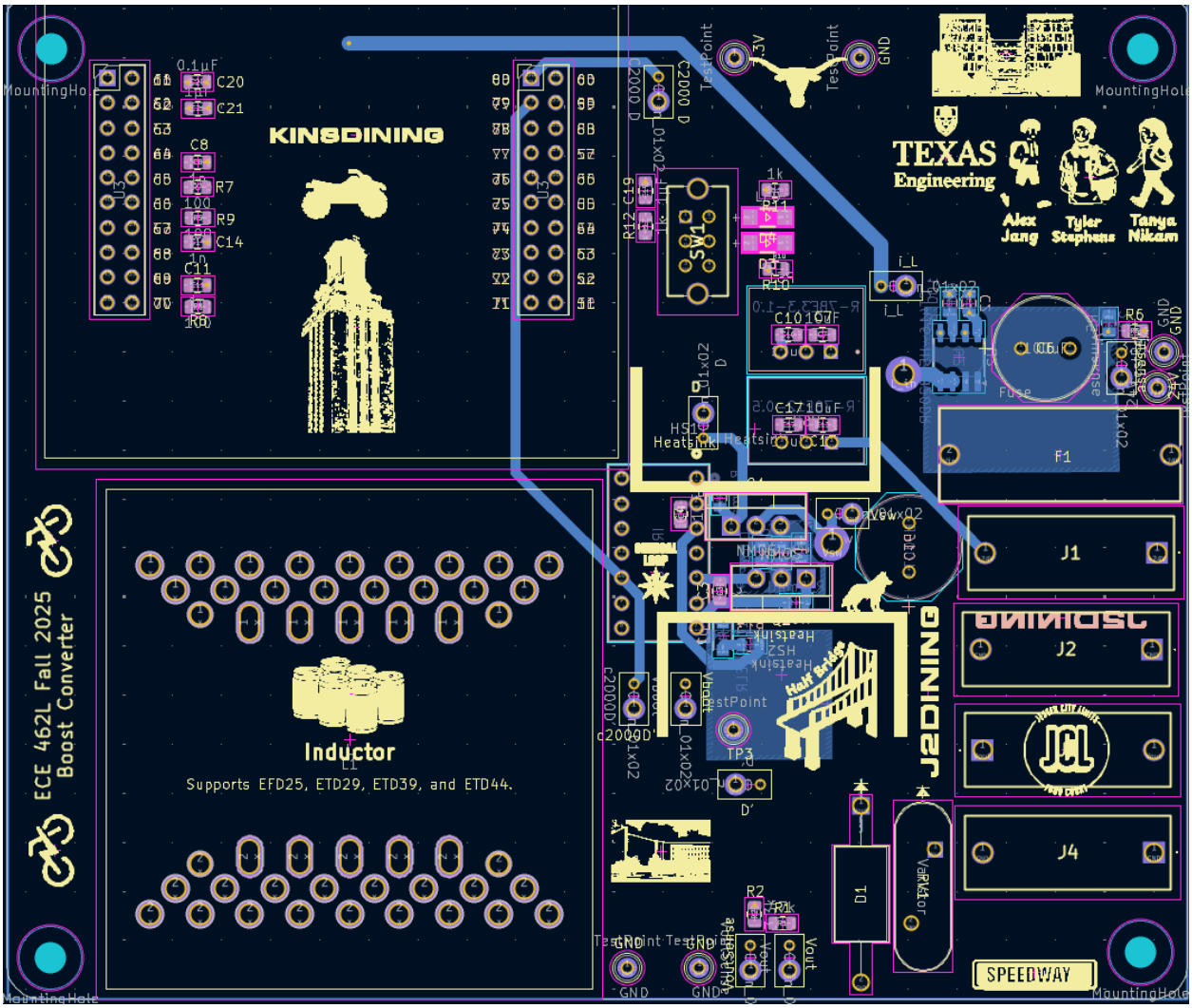

Figure 21: Bottom Layer (B.Cu)

5.3 Loop Inductance

Minimizing critical and gating loop area, shown in Figures 22, 23, and 24, was the most important consideration because it reduces the voltage spikes felt by the MOSFETs, which could risk exceeding the MOSFET's voltage rating and breaking the device. This is because a larger loop area allows for more area that magnetic flux can pass through, which increases parasitic inductance and therefore increases ringing. To reduce the loop area, we used different layers to overlap the trace paths. To fit the components on these overlapped paths, we used both the front and back side of the PCB to stack them on top of each other.

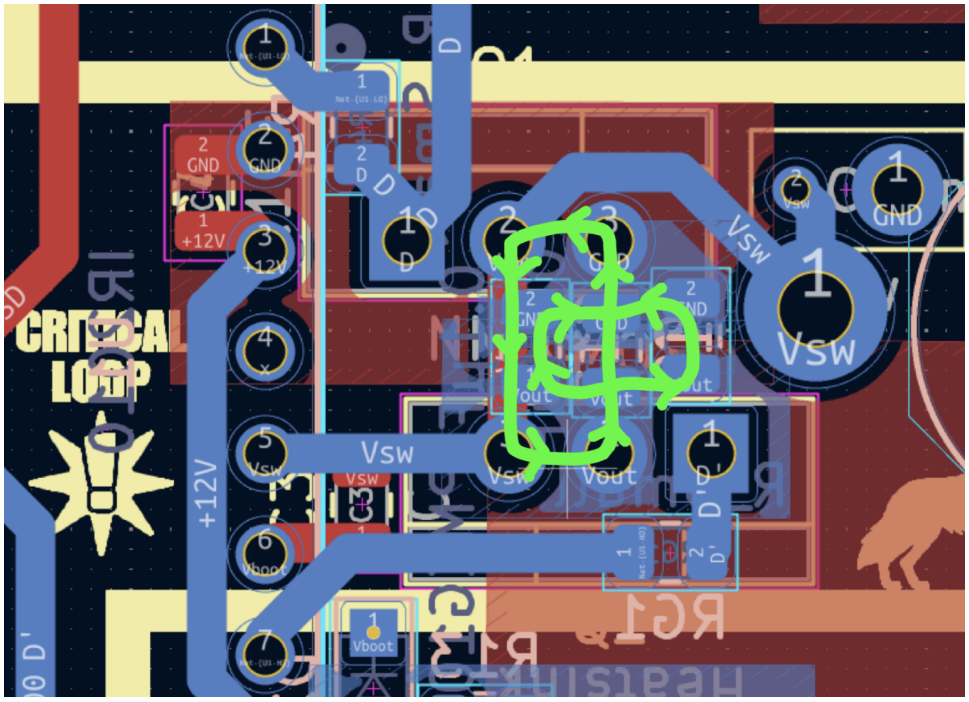

Figure 22: Critical Loop

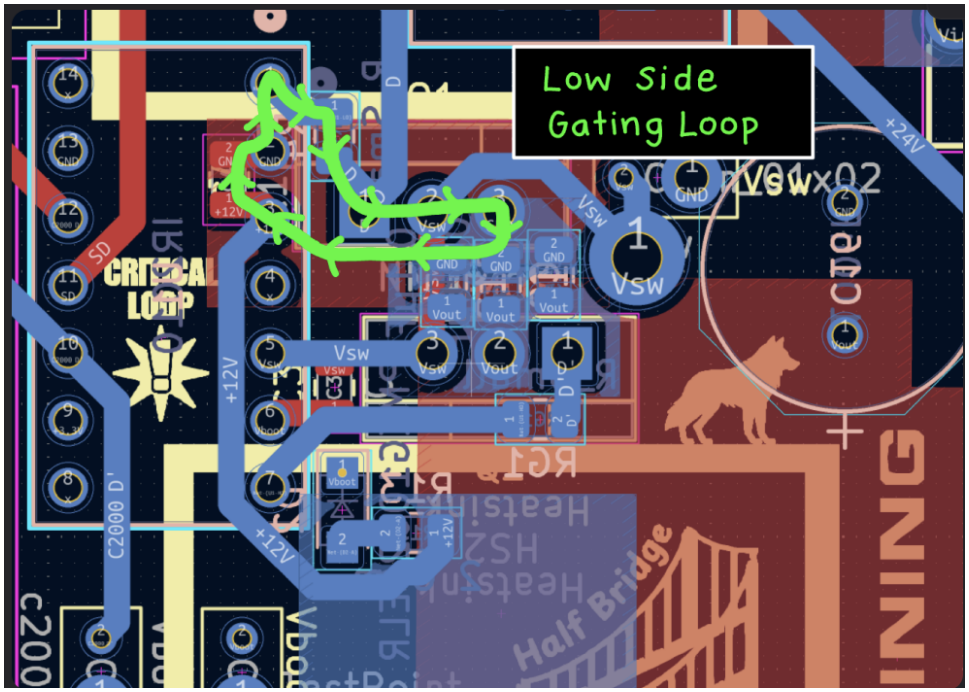

Figure 23: Low Side Gating Loop

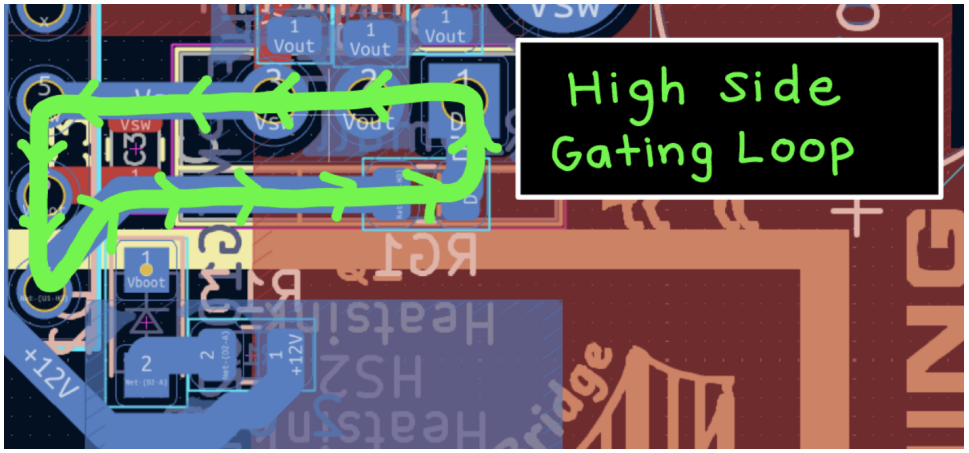

Figure 24: High Side Gating Loop

5.4 Node Capacitance

Switch node capacitance leads to greater switching losses by slowing the switching time. To minimize this, we created a cut out of the ground plane underneath our switching node to prevent capacitive coupling. This prevents the switch node and the ground plane from acting like the plates of a capacitor, reducing the parasitic capacitance and therefore improving efficiency. This design is shown below in Figure 25.

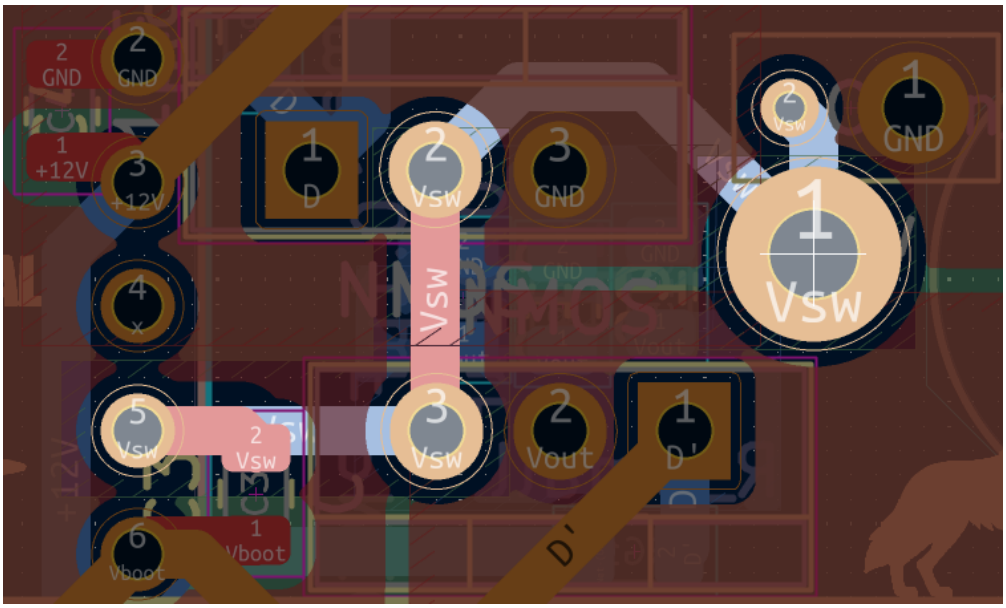

Figure 25: Switch Node and Nearby Layers

5.5 Sensitive Signals

Sensitive signals were separated from paths with high change in current to protect them from noise caused by the current fluctuations. In order to keep our sensitive signals protected, we made sure to run our traces for the ADC and PWM logic on the top left side of the board, while the return paths for the switch node current and bulk capacitor current flowed out the bottom right side of the board. This paths are shown below in Figure 26.

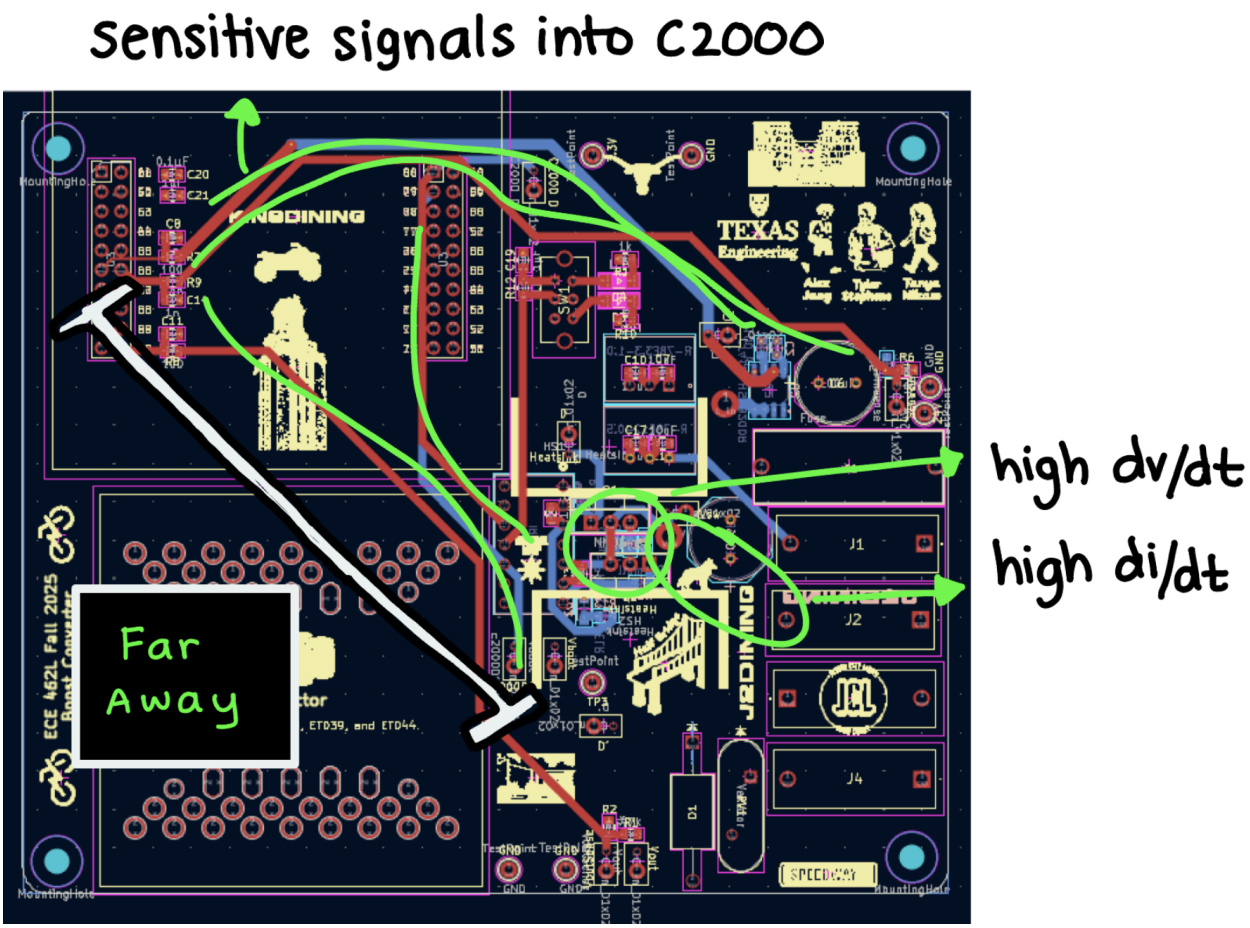

Figure 26: High dv/dt and High di/dt signals

5.6 3D Layout

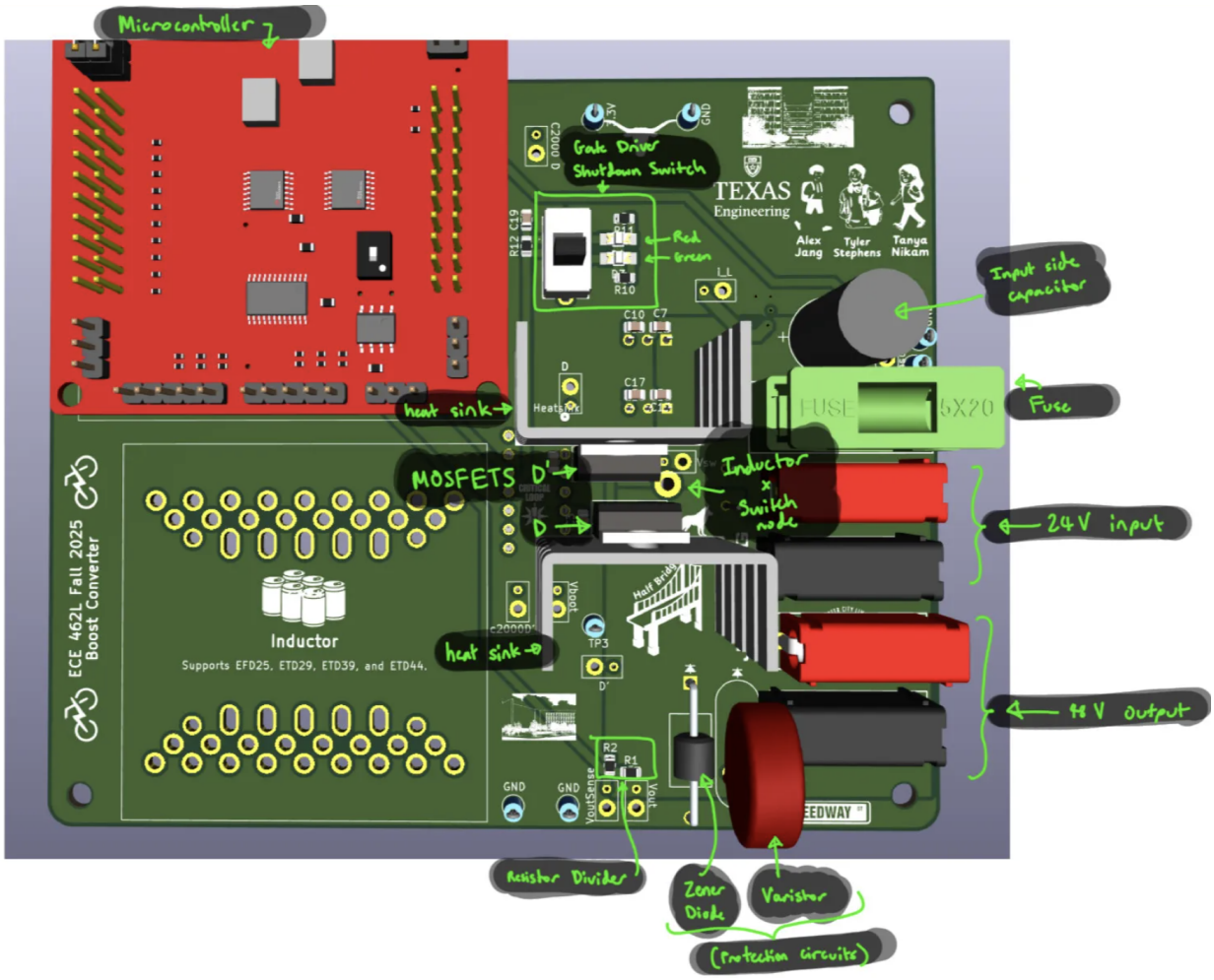

Figure 27: Labeled 3D Model of the Front Side of the PCB

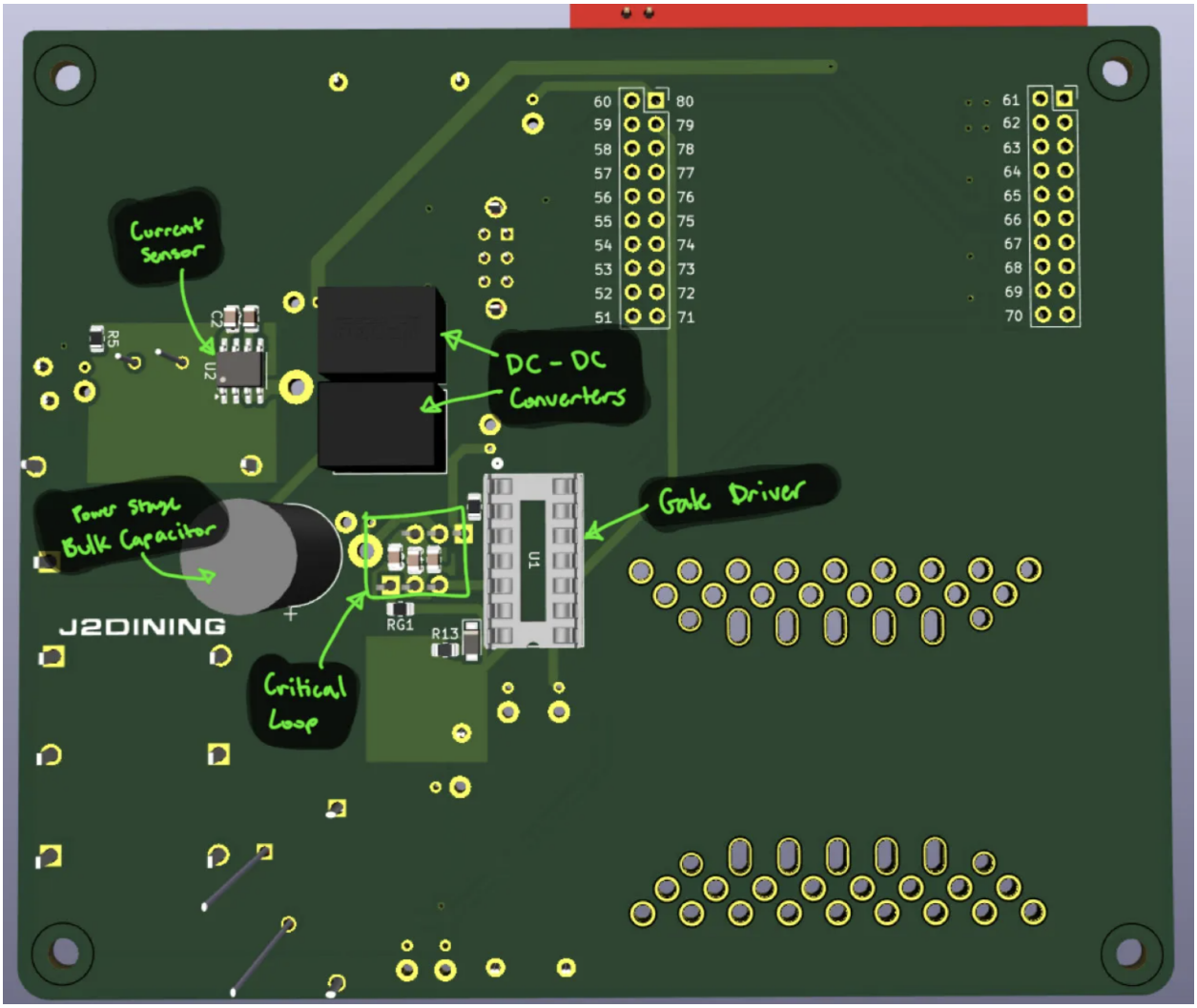

Figure 28: Labeled 3D Model of the Back Side of the PCB

In Figures 27 and 28, 3D Models of the PCB are shown. The front and back side of the PCB show what the compenents and the board would look like once everything has been soldered on.

6 Conclusion

Throughout the lab, there have been some highs and lows. One high was the PCB design itself, because it was really fun to challenge ourselves to make the loops as small as possible. All of us were familiar with routing traces and placing components, so that part was relatively simple. Another high was the fact that we got to work with a four layer board and experiment with the use of pours, which was a new learning experience for us. We also enjoyed making our design review presentation, which we count has a high because we learned a lot from other teams and our feedback. We did have some trouble with version control while sharing the latest versions of the PCB between group members, but we got that sorted out by making sure to name each version with a title that indicates what changes have been made. Finally, we had a lot of fun designing our silk screen, adding puns and campus landmarks near the different components, and we sucessfuly uploaded our board to Aisler.

Going forward, our main goal for the PCB is to maximize efficiency, even if it means sacrificing a bit on the weight and size categories. By limiting the size of our critical loop, designing large traces to support high power, and reducing parasitic R, L, and C, we have designed our PCB to maximize efficiency. Going forward, we will carefully select a MOSFET with a low on resistance and switching frequency to reduce conductive and switching losses. In addition to efficiency, our goal is to have an aesthetic board with a neat and logical layout. To keep our board neat, we placed input and output terminals close to each other and labeled all of our test points. However, we prioritized placing them closer to the signal for more accurate measurements rather than placing them on the edge for ease of access. To make our board aesthetic, we made our silkscreen into a campus map of UT, complete with the EER, the Tower, dining halls, and other fun easter eggs.

We have enjoyed designing our board so far and we are excited to select devices, build our inductor, assemble our PCB, and test the converter.

7 Appendix

7.1 Lab 2 Code

Listing 2: Lab 2 Code

```
// Included Files
//
#include "f28x_project.h"


//
// Defines
//
#define RESULTS_BUFFER_SIZE     256


//
// Globals
//
uint16_t adcAResults[RESULTS_BUFFER_SIZE];// Buffer for results
uint16_t index; // Index into result buffer
volatile uint16_t bufferFull;// Flag to indicate buffer is full
float expected_voltage;


//
// Function Prototypes
//
void initADC(void);
void initEPWM(void);
void initADCSOC(void);
__interrupt void adcA1ISR(void);


//
// Main
//
void main(void)
{
    //
    // Initialize device clock and peripherals
    //
    InitSysCtrl();


    //
    // Initialize GPIO
```

```
43      //
44      InitGpio();
45
46
47      //
48      // Disable CPU interrupts
49      //
50      DINT;
51
52
53      //
54      // Initialize the PIE control registers to their default state.
55      // The default state is all PIE interrupts disabled and flags
56      // are cleared.
57      //
58      InitPieCtrl();
59
60
61      //
62      // Disable CPU interrupts and clear all CPU interrupt flags:
63      //
64      IER = 0x0000;
65      IFR = 0x0000;
66
67
68      //
69      // Initialize the PIE vector table with pointers to the
70      // shell Interrupt
71      // Service Routines (ISR).
72      //
73      InitPieVectTable();
74
75
76      //
77      // Map ISR functions
78      //
79      EALLOW;
80      PieVectTable.ADCA1_INT = &adcA1ISR;// Function for ADCA
81      //interrupt 1
82      EDIS;
83
84
85      //
86      // Configure the ADC and power it up
87      //
88      initADC();
89
```

```
90
91      //
92      // Configure the ePWM
93      //
94      initEPWM();
95
96
97      //
98      // Setup the ADC for ePWM triggered conversions on channel 1
99      //
100     initADCSOC();
101
102
103     //
104     // Enable global Interrupts and higher priority real-time
105     //debug events:
106     //
107     IER |= M_INT1;  // Enable group 1 interrupts
108
109
110     EINT;           // Enable Global interrupt INTM
111     ERTM;           // Enable Global realtime interrupt DBGM
112
113
114     //
115     // Initialize results buffer
116     //
117     for(index = 0; index < RESULTS_BUFFER_SIZE; index++)
118     {
119         adcAResults[index] = 0;
120     }
121
122
123     index = 0;
124     bufferFull = 0;
125
126
127     //
128     // Enable PIE interrupt
129     //
130     PieCtrlRegs.PIEIER1.bit.INTx1 = 1;
131
132
133     //
134     // Sync ePWM
135     //
136     EALLOW;
```

```
137     CpuSysRegs.PCLKCR0.bit.TBCLKSYNC = 1;
138
139
140     //
141     // Take conversions indefinitely in loop
142     //
143     while(1)
144     {
145         // back calculate the voltage
146     // expected_voltage = (AdcaResultRegs.ADCRESULT0/4095.0) * 3.3;
147
148
149
150     }
151 }
152
153
154 //
155 // initADC - Function to configure and power up ADCA.
156 //
157 void initADC(void)
158 {
159     //
160     // Setup VREF as internal
161     //
162     SetVREF(ADC_ADCA, ADC_INTERNAL, ADC_VREF3P3);
163
164
165     EALLOW;
166
167
168     //
169     // Set ADCCLK divider to /4
170     //
171     AdcaRegs.ADCCTL2.bit.PRESCALE = 0;  // ADC clock same as
172     //system clock
173
174
175     //
176     // Set pulse positions to late
177     //
178     AdcaRegs.ADCCTL1.bit.INTPULSEPOS = 1;
179
180
181     //
182     // Power up the ADC and then delay for 1 ms
183     //
```

```
184     AdcaRegs.ADCCTL1.bit.ADCPWDNZ = 1;
185     EDIS;
186
187
188     DELAY_US(1000);
189 }
190
191
192 //
193 // initEPWM - Function to configure ePWM1 to generate the SOC.
194 //
195 void initEPWM(void)
196 {
197     EALLOW;
198
199
200 //never changed these in lab 1
201     EPwm1Regs.ETSEL.bit.SOCAEN = 1; // Disable SOC on A group
202     EPwm1Regs.ETSEL.bit.SOCASEL = 3;// Select SOC on up-count
203     EPwm1Regs.ETPS.bit.SOCAPRD = 1;// Generate pulse on 1st
204     //event
205     EPwm1Regs.ETPS.bit.INTPRD = 1; // interrupt on first event
206     EPwm1Regs.ETSOCPS.bit.SOCAPRD2 = 10;// every 2 events -
207     //double
208     EPwm1Regs.ETPS.bit.SOCPSSEL = 1;// range of pulese per
209     //events
210
211
212
213
214
215
216 //set based on frequency, formula is (switching freq) = 1/(2)
217 //(Nr)(Tclk)
218 //Nr = count value to plug into TBPRD, Tclk = 1/150MHz
219     EPwm1Regs.TBPRD = 375;            // Set period to 375
220     //counts (Nr)
221     EPwm1Regs.CMPA.bit.CMPA = 188;    // Set compare A value to
222     //375*(duty ratio), here duty ratio is 0.5
223     EPwm1Regs.TBCTL.bit.CLKDIV = 0;     // how fast counter
224     //should increment relative to system clock
225     EPwm1Regs.TBCTL.bit.HSPCLKDIV = 0;  //  non-divided counter
226
227
228
229
230 //Sets type of count method
```

```
231     //EPwm1Regs.TBCTL.bit.CTRMODE = 3;     // Freeze counter
232     EPwm1Regs.TBCTL.bit.CTRMODE = 2;     // up down
233     //      EPwm1Regs.TBCTL.bit.CTRMODE = 1;     // down
234     //      //EPwm1Regs.TBCTL.bit.CTRMODE = 0;     // up
235
236
237
238
239 //setting how the duty is triggered, changes based on up-down/
240 //down/up count method
241     EPwm1Regs.AQCTLA.bit.CAD = 2;//what do when down count = signal
242     EPwm1Regs.AQCTLA.bit.CAU = 1;    //what do when up count = signal
243     EPwm1Regs.AQCTLA.bit.ZRO = 0;    //what do when 0, aka do nothing
244     EPwm1Regs.AQCTLA.bit.PRD = 0;    //what do @ peak
245                                  //below is same thing but flipped
246     EPwm1Regs.AQCTLB.bit.CAD = 1;
247     EPwm1Regs.AQCTLB.bit.CAU = 2;
248     EPwm1Regs.AQCTLB.bit.ZRO = 0;
249     EPwm1Regs.AQCTLB.bit.PRD = 0;
250
251
252 //setting the GPIO bits for measuring waveform with
253 //oscilloscope, total of 4 bits
254     GpioCtrlRegs.GPAGMUX1.bit.GPIO0 = 0;     //high bits (first
255     //half) of pwm0
256     GpioCtrlRegs.GPAMUX1.bit.GPIO0 = 1;      //low bits (second
257     //half) of pwm0
258     GpioCtrlRegs.GPAGMUX1.bit.GPIO1 = 0;     //high bits (first
259     //half) of pwm0
260     GpioCtrlRegs.GPAMUX1.bit.GPIO1 = 1;      //low bits (second
261     //half) of pwm0
262
263
264
265
266
267
268 EPwm1Regs.DBCTL.bit.IN_MODE = 0;     //only shifting one either
269 //rising or falling edge, delaying both will just result in no
270 //net change
271 EPwm1Regs.DBCTL.bit.POLSEL = 2;     //inverting the b signal, 0b10
272 EPwm1Regs.DBCTL.bit.OUT_MODE = 3; //take the delay sig for both 0b11
273
274
275 EPwm1Regs.DBFED.bit.DBFED = 15;
276 EPwm1Regs.DBRED.bit.DBRED = 15;//how many clk cycles (100ns/6.67ns)
277
```

```
278
279
280
281     //WAVEFORM 2
282     //
283     //
284 //      EPwm1Regs.CMPA.bit.CMPA = 1400;// Set compare A
285 //value to 1000/2 counts
286 //      //EPwm1Regs.TBPRD = 375;// Set period to 375 counts (Nr)
287 //      EPwm1Regs.TBPRD = 2000;       // PWM2
288 //      // EPwm1Regs.TBPRD = 7500;       // PWM2
289 //      EPwm1Regs.TBCTL.bit.CLKDIV = 0;     // how fast counter
290 //should increment relative to system clock
291 //      EPwm1Regs.TBCTL.bit.HSPCLKDIV = 0;  //  non-divided counter
292
293
294
295
296 // //set to update at peak values, no up counter anymore bc
297 //instantly peak
298 //      EPwm1Regs.AQCTLA.bit.CAD = 2;
299 //      EPwm1Regs.AQCTLA.bit.CAU = 0;
300 //      EPwm1Regs.AQCTLA.bit.ZRO = 0;
301 //      EPwm1Regs.AQCTLA.bit.PRD = 1;
302 //      EPwm1Regs.AQCTLB.bit.CAD = 1;
303 //      EPwm1Regs.AQCTLB.bit.CAU = 0;
304 //      EPwm1Regs.AQCTLB.bit.ZRO = 0;
305 //      EPwm1Regs.AQCTLB.bit.PRD = 2;
306
307
308 //      GpioCtrlRegs.GPAGMUX1.bit.GPIO0 = 0;
309 //      GpioCtrlRegs.GPAMUX1.bit.GPIO0 = 1;
310 //      GpioCtrlRegs.GPAGMUX1.bit.GPIO1 = 0;
311 //      GpioCtrlRegs.GPAMUX1.bit.GPIO1 = 1;
312
313
314
315 //      //EPwm1Regs.TBCTL.bit.CTRMODE = 3;     // Freeze counter
316 //      //EPwm1Regs.TBCTL.bit.CTRMODE = 2;     // up down
317 //      EPwm1Regs.TBCTL.bit.CTRMODE = 1;     // down
318 //      //EPwm1Regs.TBCTL.bit.CTRMODE = 0;     // up
319
320
321 //
322 //WAVEFORM 3
323 //
324
```

```
325
326 // EPwm1Regs.CMPA.bit.CMPA = 4500; // Set compare A value to
327 //15000(0.3)counts
328 //      //EPwm1Regs.TBPRD = 375; // Set period to 375 counts (Nr)
329 //      //EPwm1Regs.TBPRD = 2000;       // PWM2
330 //       EPwm1Regs.TBPRD = 15000;       // PWM2
331 //      EPwm1Regs.TBCTL.bit.CLKDIV = 0;     // how fast counter
332 //should increment relative to system clock
333 //      EPwm1Regs.TBCTL.bit.HSPCLKDIV = 0;  //  non-divided counter
334
335
336
337
338 //flip from previous bc now up count is triggering, down is instant
339 //      EPwm1Regs.AQCTLA.bit.CAD = 0;
340 //      EPwm1Regs.AQCTLA.bit.CAU = 2;
341 //      EPwm1Regs.AQCTLA.bit.ZRO = 0;
342 //      EPwm1Regs.AQCTLA.bit.PRD = 1;
343 //      EPwm1Regs.AQCTLB.bit.CAD = 0;
344 //      EPwm1Regs.AQCTLB.bit.CAU = 1;
345 //      EPwm1Regs.AQCTLB.bit.ZRO = 0;
346 //      EPwm1Regs.AQCTLB.bit.PRD = 2;
347
348
349 //      GpioCtrlRegs.GPAGMUX1.bit.GPIO0 = 0;
350 //      GpioCtrlRegs.GPAMUX1.bit.GPIO0 = 1;
351 //      GpioCtrlRegs.GPAGMUX1.bit.GPIO1 = 0;
352 //      GpioCtrlRegs.GPAMUX1.bit.GPIO1 = 1;
353
354
355
356 //      //EPwm1Regs.TBCTL.bit.CTRMODE = 3;     // Freeze counter
357 //      //EPwm1Regs.TBCTL.bit.CTRMODE = 2;     // up down
358 //      //EPwm1Regs.TBCTL.bit.CTRMODE = 1;     // down
359 //      EPwm1Regs.TBCTL.bit.CTRMODE = 0;     // up
360
361
362     EDIS;
363 }
364
365
366 //
367 // initADCSOC - Function to configure ADCA's SOC0 to be
368 //triggered by ePWM1.
369 //
370 void initADCSOC(void)
371 {
```

```
372     //
373     // Select the channels to convert and the end of conversion flag
374     //
375     EALLOW;
376
377
378     AdcaRegs.ADCSOC0CTL.bit.CHSEL = 1;// SOC0 will convert pin A1
379                                       // 0:A0  1:A1
380                                       2:A2  3:A3
381                                       // 4:A4   5:A5
382                                       6:A6   7:A7
383                                       // 8:A8   9:A9
384                                       A:A10  B:A11
385                                       // C:A12  D:A13 E:A14  F:A15
386     AdcaRegs.ADCSOC0CTL.bit.ACQPS = 14;      // Sample window
387     //is 15 SYSCLK cycles
388     AdcaRegs.ADCSOC0CTL.bit.TRIGSEL = 5;    // Trigger on ePWM1 SOCA
389
390
391     AdcaRegs.ADCINTSEL1N2.bit.INT1SEL = 0; // End of SOC0 will
392     //set INT1 flag
393     AdcaRegs.ADCINTSEL1N2.bit.INT1E = 1;    // Enable
394     //INT1 flag
395     AdcaRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; // Make sure INT1
396     //flag is cleared
397
398
399     EDIS;
400 }
401
402
403 //
404 // adcA1ISR - ADC A Interrupt 1 ISR
405 //
406 __interrupt void adcA1ISR(void)
407 {
408     //
409     // Add the latest result to the buffer
410     // ADCRESULT0 is the result register of SOC0
411     adcAResults[index++] = AdcaResultRegs.ADCRESULT0;
412
413
414     // back calculate the voltage
415     expected_voltage = (AdcaResultRegs.ADCRESULT0/4095.0) * 3.3;
416
417
418     //
```

```
419     // Set the bufferFull flag if the buffer is full
420     //
421     if(RESULTS_BUFFER_SIZE <= index)
422     {
423         index = 0;
424         bufferFull = 1;
425     }
426
427
428     //
429     // Clear the interrupt flag
430     //
431     AdcaRegs.ADCINTFLGCLR.bit.ADCINT1 = 1;
432
433
434     //
435     // Check if overflow has occurred
436     //
437     if(1 == AdcaRegs.ADCINTOVF.bit.ADCINT1)
438     {
439         AdcaRegs.ADCINTOVFCLR.bit.ADCINT1 = 1; //clear INT1
440         //overflow flag
441         AdcaRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear INT1 flag
442     }
443
444
445     //
446     // Acknowledge the interrupt
447     //
448     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
449 }
450
451
452 //
453 // End of File
```